INFORMATICS INSTITUTE OF TECHNOLOGY

UNIVERSITY OF WESTMINSTER

**Informatics Institute of Technology**

Department of Computing

(B.Eng.) in Software Engineering

**Module: 5COSC007C - Object Oriented Programming**

**Module Leader: Mr. Guhanathan Poravi**

# COURSEWORK 1


Student ID : 2019448

Student UOW ID : w1761352

Student First Name: Tharindu

Student Last Name: Wickramasinghe

Tutorial Group : Group L

Full Name :- Wickramasinghege Tharindu Wickramasinghe UOW ID:- w1761352

"I confirm that I understand what plagiarism / collusion / contract cheating is and have read and understood the section on Assessment Offences in the Essential Information for Students. The work that I have submitted is entirely my own. Any work from other authors is duly referenced and acknowledged."

# Table of Contents


1) Class Diagrams and Use Case Diagrams ..... 1

1.1) Class Diagrams ..... 1

1.2) Use Case Diagrams ..... 2

1.2.1) Use case diagram for CLI ..... 2

1.2.2) Use case diagram for GUI ..... 3

2) Play Framework - Java ..... 4

2.1.1) SportsClub Class ..... 4

2.1.2) FootballClub Class ..... 5

2.1.3) UniversityFootballClub class ..... 8

2.1.4) SchoolFootballClub class ..... 9

2.1.5) Date Class ..... 10

2.1.6) Match Class ..... 12

2.2) utils ..... 13

2.2.1) DateSort class ..... 13

2.2.2)FootballClubSort class ..... 14

2.2.3) GoalsSort class ..... 15

2.2.4) WinSort class .....16

2.2.5) DateValidator Interface .....16

2.2.6) DateValidatorUsingDateFormat class .....17

2.2.6) KeyScanner class.....18

2.2.7) PremierLeagueUtils class .....20

2.3) data .....22

2.3.1) AppData class.....22

2.4) cli.....22

2.4.1) LeagueManager Interface .....22

2.4.2) PremierLeagueManager class .....23

2.4.3)PremierLeagueApplication class.....34

2.5) services .....41

2.5.1) MatchService Interface.....41

2.5.2) MatchServiceImpl class.....41

2.5.3) TeamService Interface.....45

2.5.4) TeamServiceImpl class.....45

2.6) controllers .....47

2.6.1)MatchController class.....47

2.6.2)TeamController class.....49

2.6.3)FrontendController class.....50

2.7) tasks.....52

2.7.1) AppThread class ..... 52

2.7.2) InitActorTask class ..... 53

2.7.3) InitTasksModule class ..... 54

3) Unit Testing ..... 55

3.1) TeamTest class ..... 55

3.2) TeamControllerTest class ..... 58

3.3) MatchTest class ..... 60

3.4) MatchServiceTest class ..... 63

3.5) MatchControllerTest class ..... 65

4) UI Components - Angular ..... 69

4.1)Main page (main-gui) ..... 69

4.1.1) main-gui.component.html ..... 69

4.1.2) main-gui.component.ts ..... 69

4.2)Navigation Bar ..... 69

4.2.1)app.component.html ..... 69

4.2.2) app.component.ts ..... 70

4.2.3) app-routing.module.ts ..... 71

4.3) CSS and Interfaces ..... 72

4.3.1) index.ts ..... 72

4.3.2) index.html ..... 73

4.4) View Premier League Table (first-gui) ..... 82

4.4.1) first-gui.component.html .....82

4.4.2) first-gui.component.ts .....84

4.5) Play A Random Match (second-gui) .....85

4.5.1) second-gui.component.html .....85

4.5.2) second-gui.component.ts .....86

4.5.3) second-gui-modal.component.html .....87

4.5.4) second-gui-modal.component.ts .....87

4.6) SEARCH MATCHES PLAYED WITH DATE (third-gui) .....88

4.6.1) third-gui.component.html .....88

4.6.2) third-gui.component.ts .....89

4.7) VIEW AND SORT MATCHES PLAYED WITH DATE (fourth-gui) .....91

4.7.1) fourth-gui.component.html .....91

4.7.2) fourth-gui.component.ts .....92

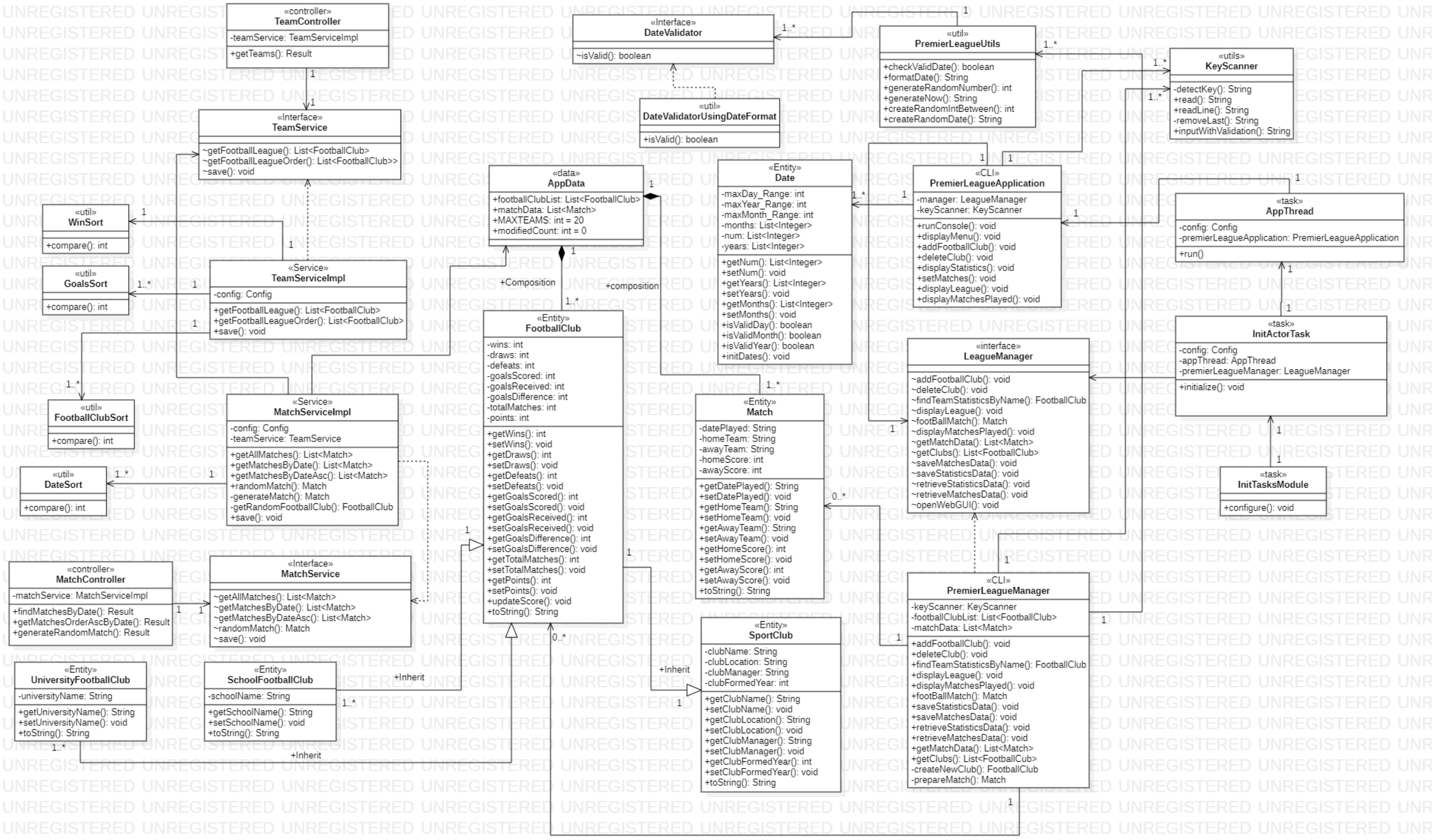
4.8) data.service.ts .....93

5) Routes – Play Framework .....94

5.1)Screenshots of using Postman to test the REST APIs .....94

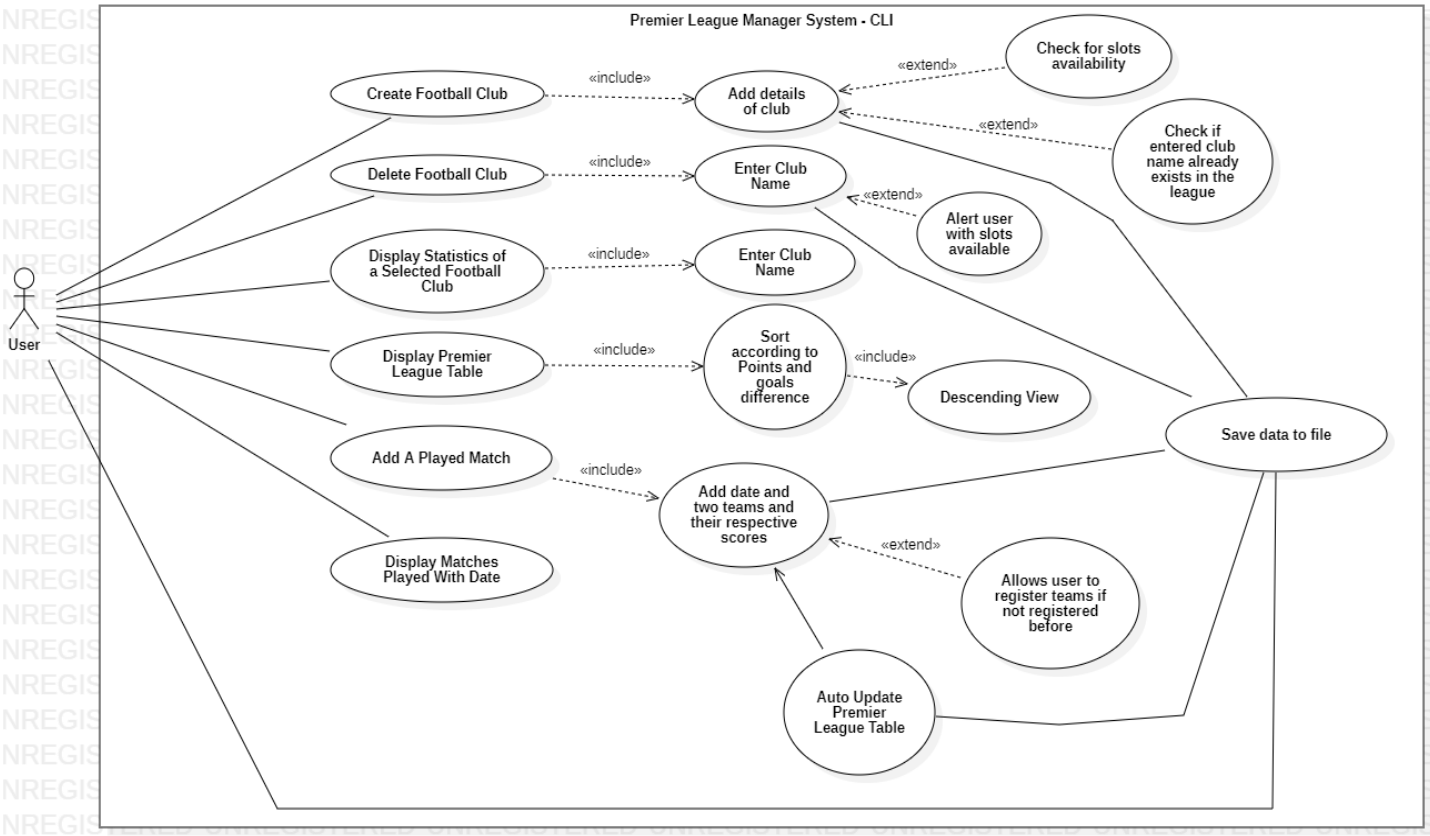
# 1) Class Diagrams and Use Case Diagrams

## 1.1) Class Diagrams



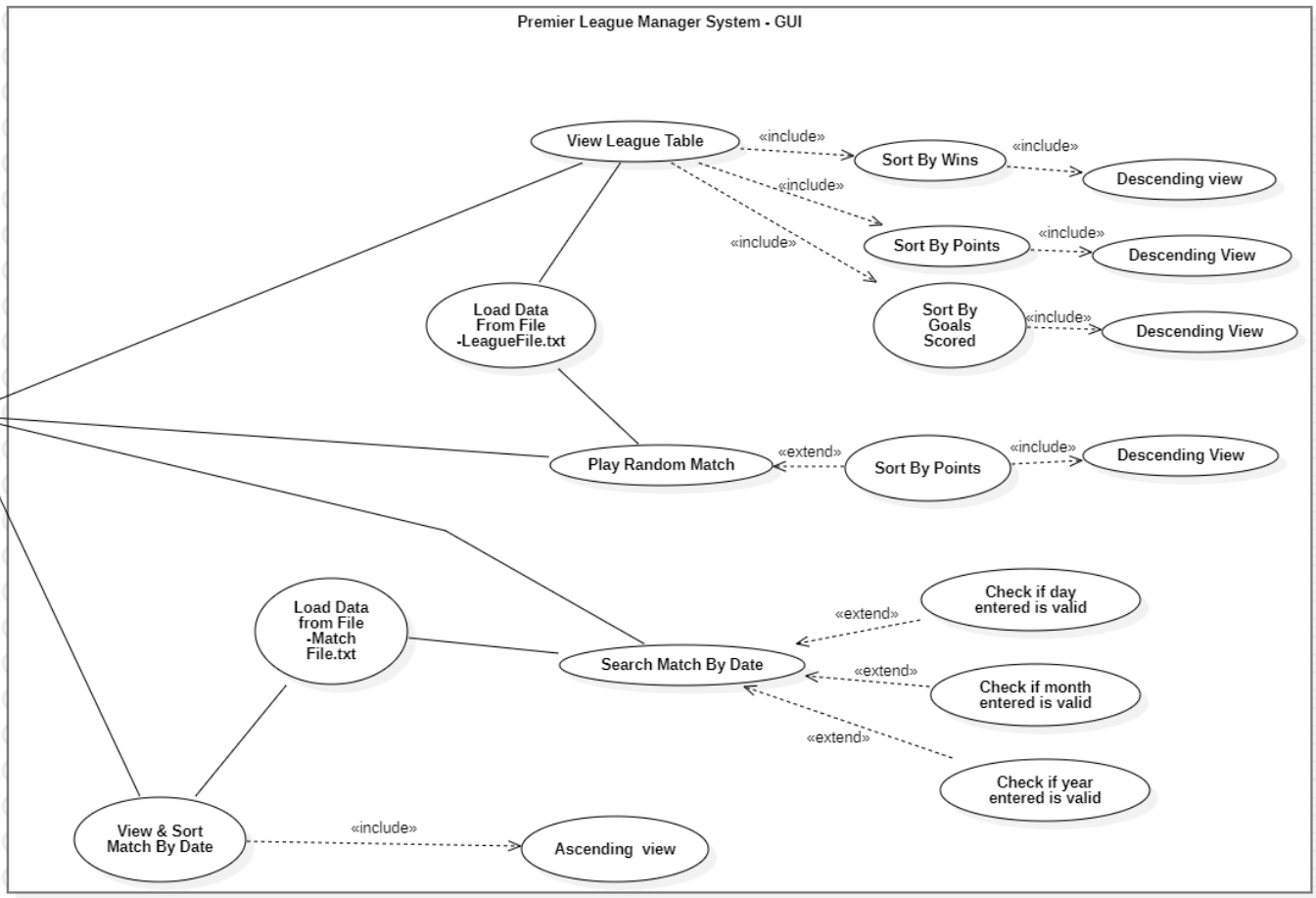
## 1.2) Use Case Diagrams

### 1.2.1) Use case diagram for CLI




UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED

### 1.2.2) Use case diagram for GUI




UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED UNREGISTERED

# 2) Play Framework - Java

### 2.1.1) SportsClub Class

```
package entities.team;

import java.io.Serializable;

/**
* Class that handle SportClub data
*/
public class SportsClub implements Serializable {
   private String clubName;
   private String clubLocation;
   private String clubManager;
   private int clubFormedYear;

   public SportsClub() {

   }

   public SportsClub(String clubName, String clubLocation, String clubManager, int
clubFormedYear) {
      this.clubName = clubName;
      this.clubLocation = clubLocation;
      this.clubManager = clubManager;
      this.clubFormedYear = clubFormedYear;

   }


   public String getClubName() {
      return clubName;
   }

   public void setClubName(String clubName) {
      this.clubName = clubName;
   }

   public String getClubLocation() {
      return clubLocation;
   }

   public void setClubLocation(String location) {
      this.clubLocation = location;
   }
```

```
    public String getClubManager() {
        return clubManager;
    }

    public void setClubManager(String clubManager) {
        this.clubManager = clubManager;
    }

    public int getClubFormedYear() {
        return clubFormedYear;
    }

    public void setClubFormedYear(int clubFormedYear) {
        this.clubFormedYear = clubFormedYear;
    }


    @Override
    public String toString() {
        return "SportsClub{clubName=" + clubName + ", clubLocation=" + clubLocation +
", clubManager='" + clubManager +
                ", clubFormedYear=" + clubFormedYear + "}";

    }
}
```

### 2.1.2) FootballClub Class

```
package entities.team;

/**
* Class that handle Football Club data
*/
public class FootballClub extends SportsClub {
    private int wins;
    private int draws;
    private int defeats;
    private int goalsScored;
    private int goalsReceived;
    private int goalsDifference;
    private int totalMatches;
    private int points;
```



```
    public FootballClub(){
```

```
    }
    public FootballClub(String name, String location,String clubManager,int
clubFormedYear) {
        super(name, location,clubManager,clubFormedYear);

    }
    public FootballClub(String name,String location,String clubManager,int
clubFormedYear,int wins,int defeats,int draws,int goalsScored,int goalsReceived,int
totalMatches,int points){
        super(name, location,clubManager,clubFormedYear);
        this.wins = wins;
        this.draws = draws;
        this.defeats = defeats;
        this.goalsScored = goalsScored;
        this.goalsReceived = goalsReceived;
        this.totalMatches = totalMatches;
        this.points = points;

    }


    public int getWins() {
        return wins;
    }

    public void setWins(int wins) {
        this.wins+= wins;
        this.points+=(wins*3);
        this.totalMatches+=wins;
    }

    public int getDraws() {
        return draws;
    }

    public void setDraws(int draws) {
        this.totalMatches+=draws;
        this.draws+= draws;
        this.points+=draws;

    }

    public int getDefeats() {
        return defeats;
    }

    public void setDefeats(int defeats) {
```

```
    this.totalMatches+=defeats;
    this.defeats =this.defeats+ defeats;

}


public int getGoalsScored() {
    return goalsScored;
}

public void setGoalsScored(int goalsScored) {
    this.goalsScored= this.goalsScored+goalsScored;
    this.goalsDifference=goalsScored-goalsReceived;


}

public int getGoalsReceived() {
    return goalsReceived;
}

public void setGoalsReceived(int goalsReceived) {

    this.goalsReceived= this.goalsReceived+goalsReceived;
    this.goalsDifference=goalsScored-goalsReceived;

}

public int getGoalsDifference() {
    return goalsDifference;
}

public void setGoalsDifference(int goalsDifference) {
    this.goalsDifference=this.goalsScored - this.goalsReceived;
}

public int getTotalMatches() {
    return totalMatches;
}

public void setTotalMatches(int totalMatches) {
    this.totalMatches = totalMatches;
}

public int getPoints() {
    return points;
}
```

```
    public void setPoints(int points) {
        this.points+= points;
    }


    public void updateScore(int goalscore, int goalreceive) {
        this.setGoalsScored(goalscore);
        this.setGoalsReceived(goalreceive);
        this.setGoalsDifference(goalscore-goalreceive);
        if (goalscore == goalreceive) {
            setDraws(1);
        }
    }

    @Override
    public String toString() {
        return  super.toString()+" FootballClub{wins=" + wins + ", draws=" + draws + ",
defeats=" + defeats + ", goalsScored=" + goalsScored +
                ", goalsReceived=" + goalsReceived + ", goalsDifference=
"+goalsDifference+", totalMatches=" + totalMatches + ", points=" + points + "}";
    }



}
```

### 2.1.3) UniversityFootballClub class

```
package entities.team;

/**
 * Class that handle UniversityFootball Club data
 */
public class UniversityFootballClub extends FootballClub{
    private String universityName;

    public UniversityFootballClub(){

    }
    public UniversityFootballClub(String universityName) {
        this.universityName = universityName;
    }

    public String getUniversityName() {
        return universityName;
```

```
    }

    public void setUniversityName(String universityName) {
        this.universityName = universityName;
    }

    @Override
    public String toString() {
        return  super.toString()+" UniversityFootballClub{universityName= " +
universityName +"}";
    }
}
```

### 2.1.4) SchoolFootballClub class

```
package entities.team;

/**
 * Class that handle SchoolFootball Club data
 */
public class SchoolFootballClub extends FootballClub{
    private String schoolName;

    public SchoolFootballClub(){

    }

    public SchoolFootballClub(String schoolName) {
        this.schoolName = schoolName;
    }

    public String getSchoolName() {
        return schoolName;
    }

    public void setSchoolName(String schoolName) {
        this.schoolName = schoolName;
    }

    @Override
    public String toString() {
        return  super.toString()+" SchoolFootballClub{schoolName= " + schoolName +"}";
    }
}
```

### 2.1.5) Date Class

```
package entities.match;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

/**
* Class that handle Date data
*/
public class Date implements Serializable {

   private static final int maxDay_Range = 30;
   private static final int maxYear_Range = 2020;
   private static final int maxMonth_Range = 12;

   private List<Integer> months = new ArrayList<>();
   private List<Integer> num = new ArrayList<>();
   private List<Integer> years = new ArrayList<>();

   public Date() {
      initDates();
   }

   public List<Integer> getNum() {
      return num;
   }

   public void setNum(List<Integer> num) {
      this.num = num;
   }

   public List<Integer> getYears() {
      return years;
   }

   public void setYears(List<Integer> years) {
      this.years = years;
   }

   public List<Integer> getMonths() {
      return months;
   }

   public void setMonths(List<Integer> months) {
      this.months = months;
   }
```

```
    public boolean isValidDay(int day) {
        int key = 0;

        if (num.contains(day)) {
            if (day <= maxDay_Range && day > 0) {
                key++;
            }
        }
        return key > 0;
    }

    public boolean isValidMonth(int str) {
        int key = 0;

        if (str <= maxMonth_Range && str > 0) {
            key++;
        }

        return key > 0;
    }

    public boolean isValidYear(int str) {
        int key = 0;
        if (years.contains(str)) {
            if (str <= maxYear_Range) {
                key++;
            }
        }
        return key > 0;
    }

    private void initDates() {
        for (int i = 0; i < 31; i++) {
            num.add(i);
        }
        for (int i = 1986; i < 2022; i++) {
            years.add(i);
        }
    }
}
```

### 2.1.6) Match Class

```
package entities.match;

import java.io.Serializable;

/**
* Class that handle Match data
*/
public class Match implements Serializable {
   private String datePlayed;
   private String homeTeam;
   private String awayTeam;
   private int homeScore;
   private int awayScore;

   public Match() {
   }

   public Match(String datePlayed, String homeTeam, String awayTeam, int
homeScore, int awayScore) {
      this.datePlayed = datePlayed;
      this.homeTeam = homeTeam;
      this.awayTeam = awayTeam;
      this.homeScore = homeScore;
      this.awayScore = awayScore;
   }


   public String getDatePlayed() {
      return datePlayed;
   }

   public void setDatePlayed(String datePlayed) {
      this.datePlayed = datePlayed;
   }

   public String getHomeTeam() {
      return homeTeam;
   }

   public void setHomeTeam(String homeTeam) {
      this.homeTeam = homeTeam;
   }

   public String getAwayTeam() {
      return awayTeam;
   }
```

```
    public void setAwayTeam(String awayTeam) {
        this.awayTeam = awayTeam;
    }

    public int getHomeScore() {
        return homeScore;
    }

    public void setHomeScore(int homeScore) {
        this.homeScore = homeScore;
    }

    public int getAwayScore() {
        return awayScore;
    }

    public void setAwayScore(int awayScore) {
        this.awayScore = awayScore;
    }


    @Override
    public String toString() {
        return "Match{datePlayed=" + datePlayed + ", homeTeam=" + homeTeam + ",
awayTeam=" + awayTeam + ", homeScore=" + homeScore +", awayScore=" +
awayScore + "}";
    }


}
```

## 2.2) utils

### 2.2.1) DateSort class

```
package utils.sort;

import entities.match.Match;

import java.text.DateFormat;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Comparator;
```

```
/**
* A Class to sort match list by date
*/
public class DateSort implements Comparator<Match> {

        /**
        * compare two match data by date
        * @param o1 Match
        * @param o2 Match
        * @return int
        */
   @Override
   public int compare(Match o1, Match o2) {
      DateFormat f = new SimpleDateFormat("dd/MM/yyyy");
      try {
         return f.parse(o1.getDatePlayed()).compareTo(f.parse(o2.getDatePlayed()));
      } catch (ParseException e) {
         throw new IllegalArgumentException(e);
      }
   }
}
```

### 2.2.2)FootballClubSort class

```
package utils.sort;

import entities.team.FootballClub;
import java.util.Comparator;

/**
* A Class to sort football club list by points
*/
public class FootballClubSort implements Comparator<FootballClub> {

        /**
        * compare 2 football club data by Points
        * @param o1 Football club
        * @param o2 Football club
        * @return int
        */
   @Override
   public int compare(FootballClub o1, FootballClub o2) {
     int rtrn;
      if(o1.getPoints() == o2.getPoints()){
```

```
            if(o1.getGoalsDifference() == o2.getGoalsDifference()){
                rtrn = 0;
            }else if(o1.getGoalsDifference() < o2.getGoalsDifference()){
                rtrn = 1;
            }else{
                rtrn = -1;
            }
        }
        else if(o1.getPoints() < o2.getPoints()){
            rtrn = 1;
        }
        else if(o1.getPoints() > o2.getPoints()){
            rtrn = -1;
        }
        else{
            rtrn = 0;
        }
        return rtrn;
    }
}
```

### 2.2.3) GoalsSort class

```
package utils.sort;

import entities.team.FootballClub;
import java.util.Comparator;

/**
* A Class to sort football club list by goals score
*/
public class GoalsSort implements Comparator<FootballClub> {

            /**
            * compare 2 football club data by goals score
            * @param ft Football club
            * @param ft1 Football club
            * @return int
            */
    @Override
    public int compare(FootballClub ft, FootballClub ft1) {
        if (ft.getGoalsScored() == ft1.getGoalsScored())
            return 0;
        else if (ft.getGoalsScored() > ft1.getGoalsScored())
            return -1;
        else
```

```
            return 1;
        }
    }
```

### 2.2.4) WinSort class

```
package utils.sort;

import entities.team.FootballClub;
import java.util.Comparator;

/**
* A Class to sort football club list by wins
*/
public class WinSort implements Comparator<FootballClub> {

            /**
             * compare 2 football club data by wins
             * @param ft Football club
             * @param ft1 Football club
             * @return int
             */
    @Override
    public int compare(FootballClub ft, FootballClub ft1) {
        if (ft.getWins() == ft1.getWins())
            return 0;
        else if (ft.getWins() > ft1.getWins())
            return -1;
        else
            return 1;
    }
}
```

### 2.2.5) DateValidator Interface

```
package utils.validator.date;

public interface DateValidator {
    boolean isValid(String dateStr);
}
```

### 2.2.6) DateValidatorUsingDateFormat class

```
package utils.validator.date;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;
import java.util.Locale;

/**
* A class that handle date validation
*/
public class DateValidatorUsingDateFormat implements DateValidator {
   private String dateFormat;

   public DateValidatorUsingDateFormat(String dateFormat) {
      this.dateFormat = dateFormat;
   }

   /**
   * check whether date valid or not
   * @param dateStr date input
   * @return boolean
   */
   @Override
   public boolean isValid(String dateStr) {
      try {
         if (dateStr.matches("([0-9]{1,2})/([0-9]{1,2})/([0-9]{4})")) {
            LocalDate.parse(dateStr,
                  DateTimeFormatter
                        .ofPattern(this.dateFormat));

         } else {
            return false;
         }
      } catch (DateTimeParseException e) {
         return false;
      }
      return true;
   }
}
```

### 2.2.6) KeyScanner class

```
package utils;

import java.io.IOException;

/**
* handle user input from console
* this class was made because CLI App runs in another Thread of Play App.
* Play Console is used by Play App to log activities of Web App.
* So, user input from another thread cannot be displayed on Play Console due to
blocking
*/
public class KeyScanner{

   /**
   * capture every keypress from user input, store it in variable
   * @param breakline option to make new line or not after enter key is pressed
   * @return String
   */
         private String detectKey(boolean breakline) throws IOException{
      String input = "";
      char c;
      int charCode = 0;

      while(true){
         charCode = System.in.read();

         if(charCode == 13){
            System.out.print("\n");
            break;
         }else if((charCode == 127) | (charCode == 8)){
            System.out.print("\b");
            input = removeLast(input);
         }else{
            c = (char) charCode;
            System.out.print(c);
            input += c;
         }

      }

      return input;
   }

   /**
   * read input user without newline
   * @return String
```

```
   */
   public String read() throws IOException{
          return detectKey(false);
   }

   /**
   * read input user with newline
   * @return String
   */
   public String readLine() throws IOException{
          return detectKey(true);
   }

   /**
   * remove last character of String
   * @param str word or String input
   * @return String
   */
   private String removeLast(String str) {
      if (str != null && str.length() > 0) {
         str = str.substring(0, str.length() - 1);
      }
      return str;
   }

   /**
   * Input with validation and error message
   * @param command
   * @param errorMessage
   * @return String
   */
   public String inputWithValidation(String command, String errorMessage) throws
IOException{
      String in = "";
      boolean err = false;

      while(in.equalsIgnoreCase("")){
         if (err) {
            System.out.println(errorMessage);
            err = false;
         }

         System.out.print(command);
         in = this.readLine();

         if (in.equalsIgnoreCase("")) {
            err = true;
         }
```

```
        }

        return in;
    }

}
```

### 2.2.7) PremierLeagueUtils class

```
package utils;

import utils.validator.date.DateValidator;
import utils.validator.date.DateValidatorUsingDateFormat;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.Random;

/**
 * Utility class for Premier League application.
 */

public final class PremierLeagueUtils {

    /**
    * check valid date
    * @param date
    * @return boolean
    */
    public static boolean checkValidDate(String date) {
        DateValidator validator = new DateValidatorUsingDateFormat("d/M/yyyy");
        return validator.isValid(date);
    }

    /**
    * change format of date, if day and month have only one character, it will be filled
with zero
    * @param date String of date
    * @return String
    */
    public static String formatDate(String date) {
        String[] split = date.split("/");
        split[0] = (split[0].length() == 1) ? "0" + split[0] : split[0];
        split[1] = (split[1].length() == 1) ? "0" + split[1] : split[1];
        return split[0] + "/" + split[1] + "/" + split[2];
    }
```

```
    /**
    * generate random number with bound
    * @param bound number of bound
    * @return int
    */
    public static int generateRandomNumber(int bound) {
        Random rand = new Random();
        return rand.nextInt(bound);
    }

    /**
    * generate random year
    * @return String
    */
    public static String generateNow() {
        return createRandomDate(2019, 2020);
    }

    /**
    * genererate random number between 2 numbers, start and end
    * @param start
    * @param end
    * @return int
    */
    public static int createRandomIntBetween(int start, int end) {
        return start + (int) Math.round(Math.random() * (end - start));
    }

    /**
    * generate random date between 2 years
    * @param startYear
    * @param endYear
    * @return String
    */
    public static String createRandomDate(int startYear, int endYear) {
        int day = createRandomIntBetween(1, 28);
        int month = createRandomIntBetween(1, 12);
        int year = createRandomIntBetween(startYear, endYear);
        return LocalDate.of(year, month, day)
                .format(DateTimeFormatter.ofPattern("dd/MM/yyyy"));
    }

}
```

## 2.3) data

### 2.3.1) AppData class

```
package data;

import entities.match.Match;
import entities.team.FootballClub;

import java.util.ArrayList;
import java.util.List;

/**
* Store Array List data of football clubs registered and data of played matches
*/
public final class AppData {
   public static List<FootballClub> footballClubList = new ArrayList<>(); //arraylist that
stores football clubs data
   public static List<Match> matchData = new ArrayList<>();  //arraylist that stores
matches data
   public static final int MAX_TEAMS = 20; //maximum number of clubs in the league
   public static int modifiedCount = 0; // This variable is used to note whether data has
modified or not by adding football club, set match or deleting club


}
```

## 2.4) cli

### 2.4.1) LeagueManager Interface

```
package cli;

import com.google.inject.ImplementedBy;
import entities.match.Match;
import entities.team.FootballClub;
import java.util.List;

@ImplementedBy(PremierLeagueManager.class)
public interface LeagueManager {
   void addFootballClub(FootballClub footballClub);

   void deleteClub(String name);

   FootballClub findTeamStatisticsByName(String name);
```

```
    void displayLeague();

    Match footBallMatch(String homeTeamName, int homeScore, String
awayTeamName, int awayScore, String dateFormat);

    void displayMatchesPlayed();

    List<Match> getMatchData();

    List<FootballClub> getClubs();

    void saveMatchesData(String filename);

    void saveStatisticsData(String filename);

    void retrieveStatisticsData(String fileName);

    void retrieveMatchesData(String fileName);

    void openWebGUI();

}
```

### 2.4.2) PremierLeagueManager class

```
package cli;

import entities.match.Match;
import entities.team.FootballClub;
import utils.PremierLeagueUtils;
import utils.sort.FootballClubSort;
import data.AppData;
import utils.KeyScanner;

import java.io.*;
import java.util.List;


/**
* This Class is responsible to manage logic of CLI application
 and to save data and retrieve data from text file
*/

public class PremierLeagueManager implements LeagueManager {
    private KeyScanner keyScanner = new KeyScanner();
    private List<FootballClub> footballClubList = AppData.footballClubList;
```

```
private List<Match> matchData = AppData.matchData;

/**
* get all match
*/
@Override
public List<Match> getMatchData() {
   return matchData;
}

/**
* get all football club
*/
@Override
public List<FootballClub> getClubs() {
   return footballClubList;
}
```

### *2.4.2.1) Creating football Club*

```
 /**
* add football club to ArrayList
* @param footballClub FootballClub instance
*/
@Override
public void addFootballClub(FootballClub footballClub) {
   if (footballClubList.isEmpty()) {
      this.footballClubList.add(footballClub);
      System.out.println("\"" + footballClub.getClubName() + "\" club is successfully
registered to the league ");
   }
            else if (this.footballClubList.size() >= AppData.MAX_TEAMS){
                  System.out.println("All the slots have been filled");
            }
            else {
      boolean findTeam = false;
      for (FootballClub footballClub1 : getClubs()) {
         if (footballClub1.getClubName().toLowerCase()
               .equals(footballClub.getClubName().toLowerCase())) {
            findTeam = true;
            break;
         }
      }
      if (!findTeam) {
         AppData.modifiedCount += 1;
         this.footballClubList.add(footballClub);
         System.out.println("\"" + footballClub.getClubName() + "\" club is successfully
registered to the league ");
```

```
                                    System.out.println("Free Slots Remaining :- " +
(AppData.MAX_TEAMS - this.footballClubList.size()));
            } else {
                System.out.println("The team is already in the list");
            }
        }
    }
```

### *2.4.2.2) Deleting football Club*

```
    /**
    * remove football club by club name
    * @param name club name
    */
    @Override
public void deleteClub(String name) {
        if (footballClubList.isEmpty()) {
            System.out.println("No teams are available in the football team list!");
        } else {
            boolean found = false;
            for (FootballClub club : footballClubList) {
                String cname = club.getClubName();
                if (cname.equals(name)) {
                    found = true;
                    footballClubList.remove(club);
                    System.out.println("\"" + name + "\" club is successfully deleted from the
list");
                    AppData.modifiedCount += 1;
          System.out.println("Free Slots Remaining :- " + (AppData.MAX_TEAMS -
this.footballClubList.size()));
                    break;
                }
            }
            if (!found) {
                System.out.println("No such club exists in the list");
            }
        }

    }
```

### *2.4.2.3) Displaying Statistics of a selected football Club*

```
    /**
    * find team statistics by name
    * @param name club name
    */
    @Override
```

```
public FootballClub findTeamStatisticsByName(String name) {
    if (footballClubList.isEmpty()) {
        System.out.println("No teams are present in the list.");
    } else {
        try {
            for (FootballClub ftc : footballClubList) {
                String cname = ftc.getClubName();
                if (cname.equals(name)) {
                    return ftc;
                }
            }
        } catch (Exception e) {
            System.out.println("Integers are not allowed");
        }
    }
    return null;
}
```

### *2.4.2.4) Displaying Premier League Table*

```
/**
* display statistics of all registered teams
*/
@Override
public void displayLeague() {
    System.out.println();
    if (footballClubList.isEmpty()) {
        System.out.println("No Club Registered To League!");
    } else {
        footballClubList.sort(new FootballClubSort());
        System.out.println("+--------------------+------------------------+---------------+-------------
-+----------------+----------------------+------------------------+-------------------------+--------------
+");
        System.out.println("|     Club Name     |   Matches Played(MP)   |   Wins(W)   |
Draws(D)  |   Defeats(L)   |   Goals Scored(GS)   |   Goals Received(GR)   |   Goals
Difference(GD)  |   Points(P)  |");
        System.out.println("+--------------------+------------------------+---------------+-------------
-+----------------+----------------------+------------------------+-------------------------+--------------
+");
        String leftAlignFormat = "| %-19s |         %-13d |     %-8d |      %-6d |       %-8d |
%-11d |          %-12d |          %-13d |       %-6d |%n";
        for (FootballClub fc : footballClubList) {
            System.out.format(leftAlignFormat,
                fc.getClubName(),
```

```
                fc.getTotalMatches(),
                fc.getWins(),
                fc.getDraws(),
                fc.getDefeats(),
                fc.getGoalsScored(),
                fc.getGoalsReceived(),
                fc.getGoalsDifference(),
                fc.getPoints());
        }
        System.out.println("-------------------------------------------------------------------------------
-------------------------------------------------------------------------------------------------------------
----------\n");
    }
  }
```

### *2.4.2.5) Adding a played match (Set a match)*

```
   /**
    * add match
   * @param teamname1 name as home team
   * @param score1 home team score
   * @param teamname2 club name as away team
   * @param score2 away team score
   * @param date date match
   * @return Match Object
   */
   private Match prepareMatch(String teamname1, int score1, String teamname2, int
score2, String date) {
      int i = 1;
      FootballClub homeClub;
      FootballClub awayClub;
      if (footballClubList.isEmpty()) {
         System.out.println("\n" + teamname1 + " isn't registered In League\n Registering
Football Club \""+teamname1 +"\" to the League ");
         homeClub = createNewClub();
         addFootballClub(homeClub);
         System.out.println("\n" + teamname2 + " isn't registered In League\n Registering
Football Club \""+teamname2 +"\" to the League ");
         awayClub = createNewClub();
         addFootballClub(awayClub);
      }
      for (FootballClub ft1 : footballClubList) {
         if (ft1.getClubName().equals(teamname1)) {
            if (score1 > score2) {
               ft1.setWins(1);
            } else if (score1 < score2) {
               ft1.setDefeats(1);
```

```
            }
            ft1.updateScore(score1, score2);
            break;
          } else if (i == footballClubList.size()) {
            if (this.footballClubList.size() >= AppData.MAX_TEAMS){
            System.out.println("All the slots have been filled");
            return null;
           }
            System.out.println("\n" + teamname1 + " isn't registered In League\n
Registering Football Club \""+teamname1 +"\" to the League ");
            homeClub = createNewClub();
            if (score1 > score2) {
               homeClub.setWins(1);

            } else if (score1 < score2) {
               homeClub.setDefeats(1);
            }
            homeClub.updateScore(score1, score2);
            addFootballClub(homeClub);
            break;
          }
          ++i;
       }
       i = 1;
       for (FootballClub ft2 : footballClubList) {
          if (ft2.getClubName().equals(teamname2)) {
            if (score1 < score2) {
               ft2.setWins(1);
            } else if (score1 > score2) {
               ft2.setDefeats(1);
            }
            ft2.updateScore(score2, score1);
            break;
          } else if (i == footballClubList.size()) {
                     if (this.footballClubList.size() >= AppData.MAX_TEAMS){
                    System.out.println("All the slots have been filled");
                    return null;
           }
            System.out.println("\n" + teamname2 + " isn't registered in League\n
Registering Football Club \""+teamname2 +"\" to the League ");
            awayClub = createNewClub();
            if (score1 < score2) {
               awayClub.setWins(1);
            }else if (score1 > score2) {
               awayClub.setDefeats(1);
            }
            awayClub.updateScore(score2, score1);
            addFootballClub(awayClub);
```

```
            break;
        }
        ++i;
    }

    AppData.modifiedCount += 1;

    Match match = new Match();
    String formatDate = PremierLeagueUtils.formatDate(date);
    match.setDatePlayed(formatDate);
    match.setHomeTeam(teamname1);
    match.setAwayTeam(teamname2);
    match.setHomeScore(score1);
    match.setAwayScore(score2);
    matchData.add(match);
    return match;
  }



   /**
  * form to entry new club
  */
  private FootballClub createNewClub() {
     try {
        FootballClub fc = new FootballClub();
        String name, location, manager;
        int yearFormed = 1900;
        name = keyScanner.inputWithValidation("Enter Club Name  : ", ">>> Club Name
cannot be empty");

        location = keyScanner.inputWithValidation("Enter Club Location  : ", ">>> Club
location cannot be empty");

        manager = keyScanner.inputWithValidation("Enter Club manager  : ", ">>> Club
manager cannot be empty");

        boolean inputValid = false;
        System.out.print("Enter Formed year (Founded year) of the Club: ");
        while (!inputValid) {
           try {
              String input = keyScanner.readLine();
              yearFormed = Integer.parseInt(input);
              inputValid = true;
           } catch (NumberFormatException e) {
              System.out.println("Only integers are allowed...Please re-enter Formed year
(Founded year) of the Club again: ");
```

```
            }
        }

        fc.setClubName(name.trim());
        fc.setClubManager(manager.trim());
        fc.setClubFormedYear(yearFormed);
        fc.setClubLocation(location.trim());
        return fc;
      } catch (IOException e) {
        System.out.println(e.toString());
        return null;
      } catch (Exception e) {
        System.out.println(e.toString());
        return null;
      }
    }



    /**
    * add match
    * @param homeTeamName club name as home team
    * @param homeScore home team score
    * @param awayTeamName club name as away team
    * @param awayScore away team score
    * @param dateFormat date match
    * @return Match Object
    */
    @Override
    public Match footBallMatch(String homeTeamName, int homeScore, String
awayTeamName, int awayScore, String dateFormat) {
        return prepareMatch(homeTeamName, homeScore, awayTeamName, awayScore,
dateFormat);
    }
```

### *2.4.2.5) Display All Matches Played*

```
/**
* Displays All Matches Played with date
*/
    @Override
    public void displayMatchesPlayed() {
        System.out.println("\nDiplaying All Played Matches\n");
        if (matchData.isEmpty()) {
            System.out.println("No Matches Played !");
```

```
    } else {
        for (Match mt : matchData) {
            System.out.printf("\nDate: %s\n%s    vs    %s\n%s     :     %s ",
mt.getDatePlayed(), mt.getHomeTeam(), mt.getAwayTeam(), mt.getHomeScore(),
mt.getAwayScore());
            System.out.println();
        }
    }
    System.out.println();
  }
```

### *2.4.2.6) Save Data to file*

```
  /**
   * save all football club data from ArrayList and put it into txt file
   * @param fileName file contains football club data
   */
  @Override
  public void saveStatisticsData(String fileName) {
    if (AppData.modifiedCount > 0) {
      try {
        FileOutputStream fileOutputStream = new FileOutputStream(fileName);
        ObjectOutputStream objectOutputStream = new
ObjectOutputStream(fileOutputStream);

        for (FootballClub footballClub : footballClubList) {
          objectOutputStream.writeObject(footballClub);
        }

        objectOutputStream.flush();
        fileOutputStream.close();
        objectOutputStream.close();
        System.out.println("Statistics Data has been saved successfully");
      } catch (IOException e) {
        System.out.println("No statistics data present to save in file");
      }
    }else{
      System.out.println("No statistics data present to save in file");
    }

  }
```



```
  /**
```

```
    * save all match data from ArrayList and put it into txt file
    * @param fileName file contains match data
    */
    @Override
    public void saveMatchesData(String fileName) {
        if (AppData.modifiedCount > 0) {
            try {
                FileOutputStream fileOutputStream = new FileOutputStream(fileName);
                ObjectOutputStream objectOutputStream = new
ObjectOutputStream(fileOutputStream);

                for (Match match : matchData) {
                    objectOutputStream.writeObject(match);
                }

                objectOutputStream.flush();
                fileOutputStream.close();
                objectOutputStream.close();
                System.out.println("Matches Data has been saved successfully");
            } catch (IOException e) {
                System.out.println("No matches played data present to save in file");
            }
        }else{
            System.out.println("No matches played data present to save in file");
        }

    }
```

### *2.4.2.7) Load Data from File*

```
     /**
    * load all football club data from file and put it into ArrayList
    * @param fileName file contains footballclub data
    */
    @Override
    public void retrieveStatisticsData(String fileName) {
        footballClubList.clear();
        try {
            FileInputStream fileInputStream = new FileInputStream(fileName);
            ObjectInputStream objectInputStream = new
ObjectInputStream(fileInputStream);
            for (; ; ) {
                try {
                    Object readObject = objectInputStream.readObject();
                    if (readObject instanceof FootballClub) {
                        footballClubList.add((FootballClub) readObject);
                    }
```

```
            } catch (EOFException e) {
                break;
            }
        }
        fileInputStream.close();
        objectInputStream.close();
        System.out.println("Statistics Data has been loaded successfully");
    } catch (IOException | ClassNotFoundException e) {
        System.out.println("No saved records present to load from file");
    }
}



/**
* load all match data from file and put it into ArrayList
* @param fileName file contains match data
*/
@Override
public void retrieveMatchesData(String fileName) {
    matchData.clear();
    try {
        FileInputStream fileInputStream = new FileInputStream(fileName);
        ObjectInputStream objectInputStream = new
ObjectInputStream(fileInputStream);
        for (; ; ) {
            try {
                Object readObject = objectInputStream.readObject();
                if (readObject instanceof Match) {
                    matchData.add((Match) readObject);
                }
            } catch (EOFException e) {
                break;
            }
        }
        fileInputStream.close();
        objectInputStream.close();
        System.out.println("Matches played Data has been loaded successfully");
    } catch (IOException | ClassNotFoundException e) {
        System.out.println("No saved records present to load from file");
    }
}
```

#### *2.4.2.8) View Premier League GUI*

```
/**
* Open browser as app GUI automatically with address host:4200
*/
@Override
public void openWebGUI() {
   System.out.println("Please Wait...Opening Premier League GUI");
   String url = "http://localhost:4200";
   String os = System.getProperty("os.name").toLowerCase();
   Runtime rt = Runtime.getRuntime();
   try {
      if (os.contains("win")) {   //for windows
         rt.exec("rundll32 url.dll,FileProtocolHandler " + url);

      } else if (os.contains("mac")) {   //for mac os
         rt.exec("open " + url);

      } else if (os.contains("nix") || os.contains("nux")) {   // for linux
         rt.exec("xdg-open " + url);
      }
   } catch (Exception ignored) {
   }
}
```

### 2.4.3)PremierLeagueApplication class

```
package cli;

import entities.match.Date;
import entities.team.FootballClub;
import utils.KeyScanner;
import data.AppData;

import java.util.Objects;
import java.io.IOException;

/**
* Class to run CLI Application
*
*/
public class PremierLeagueApplication {
   private LeagueManager manager = new PremierLeagueManager();
   private KeyScanner keyScanner = new KeyScanner();

   /**
   * Run console app
   * @param leagueFile path file / filename that store Football Club data
```

```
   * @param matchFile path file / filename that store Match data
   */
   public void runConsole(String leagueFile, String matchFile) throws IOException {
      manager.retrieveStatisticsData(leagueFile);
      manager.retrieveMatchesData(matchFile);

System.out.println("============================================================
==============================\n");
      System.out.println("**~**********~**********~** - WELCOME TO PREMIER LEAGUE
APPLICATION -  *~**********~**********~**");


      boolean showMenu = true;
      while (showMenu) {
         displayMenu();

         String choice = keyScanner.readLine();

         switch (choice) {
            case "1":
               addFootballClub();
               break;
            case "2":
               deleteClub();
               break;
            case "3":
               displayStatistics();
               break;
            case "4":
               displayLeague();
               break;
            case "5":
               setMatches();
               break;
            case "6":
               displayMatchesPlayed();
               break;
            case "7":
               manager.saveStatisticsData(leagueFile);
               manager.saveMatchesData(matchFile);
               AppData.modifiedCount = 0;
               break;
            case "8":
               manager.openWebGUI();
               break;
            case "0":
               showMenu = false;
               System.out.println("Thank you for using the system.....");
```

```
                break ;
            default:
                System.out.println("Invalid option entered!!!....Please re-enter");
                break;
        }
    }
  }

  /**
  * Display menu option for Premier League CLI App.
  */
  private void displayMenu(){

System.out.println("\n==========================================================
==================================\n");
      System.out.println("Enter (1) - Register (Create) A Club");
      System.out.println("Enter (2) - Delete A Club");
      System.out.println("Enter (3) - Display Statistics of a Selected Club ");
      System.out.println("Enter (4) - Display PremierLeague Table");
      System.out.println("Enter (5) - Add a Played Match with date (Set a Match) ");
System.out.println("Enter (6) - Display All Matches Played ");
      System.out.println("Enter (7) - Save Data to File ");
      System.out.println("Enter (8) - Open Premier League Web GUI ");
      System.out.println("Enter (0) - Quit");

System.out.println("==========================================================
==============================\n");
      System.out.println("> Please Select An Option From The Following");
  }



  /**
  * Add football club
  */
  private void addFootballClub() throws IOException{
      FootballClub footballClub;
      String clubName = keyScanner.inputWithValidation("\nEnter Club  : ", ">>> Club
cannot be empty");

      String clubLoc = keyScanner.inputWithValidation("\nEnter Club Location: ", ">>>
Club location cannot be empty");

      String managerName = keyScanner.inputWithValidation("\nEnter name of the Club
Manager: ", ">>> Club manager cannot be empty");
```



```
      boolean inputValid = false;
      System.out.print("\nEnter Formed year (Founded year) of the Club: ");
```

```
      while (!inputValid) {
         try {
            String input = keyScanner.readLine();
            int formedYear = Integer.parseInt(input);
            inputValid = true;

            footballClub = new FootballClub(clubName.trim(), clubLoc.trim(),
managerName.trim(), formedYear);
            manager.addFootballClub(footballClub);
         } catch (NumberFormatException e) {
            System.out.println("Only integers are allowed...Please re-enter Formed year
(Founded year) of the Club again: ");
         }
      }


   }

   /**
   * Delete football club
   */
   private void deleteClub() throws IOException {
      String name = keyScanner.inputWithValidation("\nPlease enter club name you
want to delete : ", ">>> Club name cannot be empty");
      manager.deleteClub(name.trim());
   }

   /**
   * Display football club statistics
   */
   private void displayStatistics() {
      boolean inputValid = false;
      System.out.println("\nEnter Name of Club to display statistics: ");
      while (!inputValid) {
         try {
            String name = keyScanner.inputWithValidation("Enter Club Name  : ", ">>>
Club Name cannot be empty");
            FootballClub ftc = manager.findTeamStatisticsByName(name.trim());
            if (Objects.isNull(ftc)) {
               System.out.println("No such club exists");
               break;
            } else {
               System.out.printf("\nClub Name: %s\nClub Location: %s\nClub Manager:
%s\nClub Formed Year: %s\nMatches played: %s\nWins: %s\nDraws: %s\nDefeat:
%s\nGoals Scored: %s\nGoals Received: %s\nGoals Difference: %s\nPoints: %s\n",
ftc.getClubName(), ftc.getClubLocation(),ftc.getClubManager(),
ftc.getClubFormedYear(),ftc.getTotalMatches(), ftc.getWins(), ftc.getDraws(),
ftc.getDefeats(), ftc.getGoalsScored(), ftc.getGoalsReceived(), ftc.getGoalsDifference(),
```

```
ftc.getPoints());
                System.out.println();
                inputValid = true;
            }

        } catch (Exception e) {
            System.out.println("No integers are allowed...Please re-enter name of the club
to display statistics ");
        }
    }

  }

  /**
  * Set Match by entry home team and away team. If teams that are entered is not
registered,
  * App will open prompt to add football club
  */
  private void setMatches() throws IOException {
    Date date = new Date();
    System.out.println("\nSetting Matches...\n");
    String homeTeamName, awayTeamName;
    int homeScore = -1, awayScore = -1;
    System.out.print("Enter Date Of Match(DD/MM/YYY)");
    int year, month, day;
    String dateFormat = "";
    boolean inputValidDay = false, inputValidMonth = false, inputValidYear = false,
        inputValidHomeScore = false, inputValidAwayScore = false;

    while (!inputValidDay) {
      try {
        System.out.print("\nDay: ");
        String dd = keyScanner.readLine();
        day = Integer.parseInt(dd);
        if (date.isValidDay(day)) {
          dateFormat += "" + day;
          inputValidDay = true;
        } else {
          System.out.println("Invalid Day Input!");
        }
      } catch (NumberFormatException e) {
        System.out.println("Invalid Day Input!");
      }
    }
```



```
    while (!inputValidMonth) {
      try {
```

```
            System.out.print("Month: ");
            String mm = keyScanner.readLine();

            month = Integer.parseInt(mm);
            if (date.isValidMonth(month)) {
               dateFormat += "/" + month + "/";
               inputValidMonth = true;
            } else {
               System.out.println("Invalid Month Input!");
            }
         } catch (NumberFormatException e) {
            System.out.println("Invalid Month Input!");
         }
      }

      while (!inputValidYear) {
         try {
            System.out.print("Year: ");
            String yy = keyScanner.readLine();

            year = Integer.parseInt(yy);
            if (date.isValidYear(year)) {
               dateFormat += "" + year;
               inputValidYear = true;
            } else {
               System.out.println("Invalid Year Input!");
            }
         } catch (NumberFormatException e) {
            System.out.println("Invalid Year Input!");
         }
      }

      System.out.println();

      System.out.print("Home Team Setup\n");
      homeTeamName = keyScanner.inputWithValidation("Enter Club Name  : ", ">>>
Club Name cannot be empty");

      while (!inputValidHomeScore) {
         try {
            System.out.print("Enter Club Score: ");
            String scoreHome = keyScanner.readLine();

            homeScore = Integer.parseInt(scoreHome);
            if (homeScore >= 0) {
               inputValidHomeScore = true;
            } else {
               System.out.println("Invalid Score Input!");
```

```
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid Score Input!");
        }
    }

    System.out.println();

    System.out.print("Away Team Setup\n");
    awayTeamName = keyScanner.inputWithValidation("Enter Club Name  : ", ">>>
Club Name cannot be empty");

    while (!inputValidAwayScore) {
        try {
            System.out.print("Enter Club Score: ");
            String scoreAway = keyScanner.readLine();

            awayScore = Integer.parseInt(scoreAway);
            if (awayScore >= 0) {
                inputValidAwayScore = true;
            } else {
                System.out.println("Invalid Score Input!");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid Score Input!");
        }
    }

    System.out.println("\nMatch has been added successfully");
    manager.footBallMatch(homeTeamName.trim(), homeScore,
awayTeamName.trim(), awayScore, dateFormat);
  }

  /**
  * Display football club list with their statistics
  */
  private void displayLeague()  {
    manager.displayLeague();
  }

  /**
  * Display all match
  */
  private void displayMatchesPlayed(){
    manager.displayMatchesPlayed();
  }
}
```

## 2.5) services

### 2.5.1) MatchService Interface

```
package services.match;

import com.google.inject.ImplementedBy;

import entities.match.Match;

import java.util.List;

@ImplementedBy(MatchServiceImpl.class)
public interface MatchService {
   List<Match> getAllMatches();

   List<Match> getMatchesByDate(String date);

   List<Match> getMatchesByDateAsc();

   Match randomMatch();

   void save(Match match);
}
```

### 2.5.2) MatchServiceImpl class

```
package services.match;

import entities.match.Match;
import entities.team.FootballClub;
import utils.sort.DateSort;
import utils.PremierLeagueUtils;
import data.AppData;
import services.team.TeamService;

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import javax.inject.Inject;
import com.typesafe.config.Config;

/**
```

```
* Class that handles Match data
*/
public class MatchServiceImpl implements MatchService {
   private final Config config;
   private final TeamService teamService;

   @Inject
   public MatchServiceImpl(Config config, TeamService teamService) {
      this.config = config;
      this.teamService = teamService;
   }

   /**
   * get all match data
   * @return {@code List<MatchData>}
   */
   @Override
   public List<Match> getAllMatches() {
      return AppData.matchData;
   }

   /**
   * get match data by date
   * @param date
   * @return {@code List<MatchData>}
   */
   @Override
   public List<Match> getMatchesByDate(String date) {
      List<Match> allMatches = this.getAllMatches();
      List<Match> allMatchesByDate = new ArrayList<>();
      for (Match match : allMatches) {
         String formatDate = PremierLeagueUtils.formatDate(date);
         if (formatDate.equals(match.getDatePlayed())) {
            allMatchesByDate.add(match);
         }
      }
      return allMatchesByDate;
   }

   /**
   * get all match data order by date ascending
   * @return {@code List<MatchData>}
   */
   @Override
   public List<Match> getMatchesByDateAsc() {
      List<Match> all = this.getAllMatches();
      all.sort(new DateSort());
      return all;
```

```
    }

    /**
    * generate random Match
    * @return {@code MatchData}
    */
    @Override
    public Match randomMatch() {
        FootballClub home = getRandomFootballClub();
        FootballClub away = getRandomFootballClub();
        while (home.getClubName().equals(away.getClubName())) {
            away = getRandomFootballClub();
        }
        return generateMatch(home, away);
    }

    /**
    * generate random Match process
    * @return {@code MatchData}
    */
    private Match generateMatch(FootballClub home, FootballClub away) {
        int goalsScoredHome = PremierLeagueUtils.generateRandomNumber(6);
        int goalsScoredAway = PremierLeagueUtils.generateRandomNumber(6);
        if (goalsScoredHome < goalsScoredAway) {
            // Away Win
            home.setDefeats(1);
            away.setWins(1);
            away.updateScore(goalsScoredAway, goalsScoredHome);
            home.updateScore(goalsScoredHome, goalsScoredAway);

        } else if (goalsScoredHome > goalsScoredAway) {
            // Home Win
            away.setDefeats(1);
            home.setWins(1);
            home.updateScore(goalsScoredHome, goalsScoredAway);
            away.updateScore(goalsScoredAway, goalsScoredHome);
        }
                else {
            // Match Draw
            home.updateScore(goalsScoredHome, goalsScoredAway);
            away.updateScore(goalsScoredAway, goalsScoredHome);
        }
        Match match = new Match(PremierLeagueUtils.generateNow(),
home.getClubName(), away.getClubName(), goalsScoredHome, goalsScoredAway);
        this.save(match);
        teamService.save(home);
        teamService.save(away);
        return match;
```

```
    }

    /**
    * generate random football club
    * @return {@code FootballClub}
    */
    private FootballClub getRandomFootballClub() {
        List<FootballClub> footballClubList = teamService.getFootballLeague();
        return
footballClubList.get(PremierLeagueUtils.generateRandomNumber(footballClubList.size(
)));
    }

    /**
    * save all match data from ArrayList and put it into txt file
    * @param match file contains match data
    */
    @Override
    public void save(Match match) {
        List<Match> allMatches = this.getAllMatches();
        allMatches.add(match);
        try {
            String matchFile = config.getString("file.match_file_name");
            FileOutputStream fileOutputStream = new FileOutputStream(matchFile);
            ObjectOutputStream objectOutputStream = new
ObjectOutputStream(fileOutputStream);

            allMatches.forEach(el -> {
                try {
                    objectOutputStream.writeObject(el);
                } catch (IOException e) {
                    e.printStackTrace();
                }
            });
            fileOutputStream.close();
            objectOutputStream.close();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

}
```

### 2.5.3) TeamService Interface

```
package services.team;

import com.google.inject.ImplementedBy;

import entities.team.FootballClub;

import java.util.List;

@ImplementedBy(TeamServiceImpl.class)
public interface TeamService {
   List<FootballClub> getFootballLeague();

   List<FootballClub> getFootballLeagueOrder(String orderBy);

   void save(FootballClub footballClub);
}
```

### 2.5.4) TeamServiceImpl class

```
package services.team;

import entities.team.FootballClub;
import utils.sort.FootballClubSort;
import utils.sort.GoalsSort;
import utils.sort.WinSort;
import data.AppData;
import java.io.*;
import java.util.List;
import java.util.ListIterator;
import javax.inject.Inject;
import com.typesafe.config.Config;

/**
* Class that handles Football Club data
*/
public class TeamServiceImpl implements TeamService {

   private final Config config;

   @Inject
   public TeamServiceImpl(Config config) {
      this.config = config;
```

```
    }

    /**
    * get all football club
    * @return {@code List<FootballClub>}
    */
    @Override
    public List<FootballClub> getFootballLeague() {
        return AppData.footballClubList;
    }

    /**
    * get football club order by option descending
    * @param orderBy sort option (goals, wins, points)
    * @return {@code List<FootballClub>}
    */
    @Override
    public List<FootballClub> getFootballLeagueOrder(String orderBy) {
        List<FootballClub> footballClubList = this.getFootballLeague();
        switch (orderBy) {
            case "goals":
                footballClubList.sort(new GoalsSort());
                break;
            case "wins":
                footballClubList.sort(new WinSort());
                break;
            case "points":
                footballClubList.sort(new FootballClubSort());
                break;
        }
        return footballClubList;
    }

    /**
    * save all football club data from ArrayList and put it into txt file
    * @param footballClub file contains football club data
    */
    @Override
    public void save(FootballClub footballClub) {
        List<FootballClub> allTeams = this.getFootballLeague();
        ListIterator<FootballClub> footballClubListIterator = allTeams.listIterator();
        while (footballClubListIterator.hasNext()) {
            if
(footballClubListIterator.next().getClubName().equals(footballClub.getClubName())) {
                footballClubListIterator.set(footballClub);
            }
        }
        try {
```

```
            String leagueFile = config.getString("file.league_file_name");
            FileOutputStream fileOutputStream = new FileOutputStream(leagueFile);
            ObjectOutputStream objectOutputStream = new
    ObjectOutputStream(fileOutputStream);

            allTeams.forEach(el -> {
               try {
                  objectOutputStream.writeObject(el);
               } catch (IOException e) {
                  e.printStackTrace();
               }
            });
            fileOutputStream.close();
            objectOutputStream.close();

         } catch (IOException e) {
            e.printStackTrace();
         }
      }

    }
```

## 2.6) controllers

### 2.6.1)MatchController class

```
/**
* Cannot use "match" keyword because in Scala (the main language that builds the play
framework Play Framework)
* "match" keyword is reverse keyword
*
*/

package controllers.matches;

import javax.inject.Inject;

import play.mvc.*;
import play.libs.Json;

import services.match.MatchService;
import entities.match.Match;
import utils.PremierLeagueUtils;

import java.util.List;
```



```
/**
* Class that handle MatchesPlayed API data
```

```
*/
public class MatchController extends Controller {
        private final MatchService matchService;

        @Inject
        public MatchController(MatchService matchService){
                this.matchService = matchService;
        }

  /**
   * To Find a match by date.The result is {@code List<Match>} with all matches in the
given date {@code String date}
   * path = "/matches"
   *
   * @param date (default value = "") date parameter to get matches
   * @return {@code List<Match>} or ResponseEntity no content in case of no results
   *
   */
  public Result findMatchesByDate(String date) {
    if (!date.isEmpty() && !PremierLeagueUtils.checkValidDate(date)) {
      return status(409, "Malformed date");
    }
    try {
      List<Match> matches = (date.isEmpty()) ? matchService.getAllMatches() :
matchService.getMatchesByDate(date);
      if (matches.size() > 0) {
        return ok(Json.toJson(matches));
      } else {
        System.out.println("no content");
        return noContent();
      }
    } catch (Exception ex) {
      ex.printStackTrace();
      return badRequest();
    }
  }

  /**
   * Gets all matches in ascending order by date. The result is {@code List<Match>}
with all matches order by date
   * path = "/matches/order-by-date"
   *
   * @return {@code List<Match>} order descending date or ResponseEntity no
content in case of no results.
   */
  public Result getMatchesOrderAscByDate() {
    try {
      List<Match> matches = matchService.getMatchesByDateAsc();
```

```
            if (matches.size() > 0) {
                return ok(Json.toJson(matches));
            } else {
                return noContent();
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            return badRequest();
        }
    }


     /**
     * Start a match between two randoms {@code FootballClub} teams the result is a
{@code Match}
     * otherwise a bad request is returned.
     * path = "/random"
     *
     * @return Random {@code Match} or ResponseEntity bad request.
     */
    public Result generateRandomMatch() {
        Match randomMatch = matchService.randomMatch();
        try {
            return ok(Json.toJson(randomMatch));
        } catch (Exception ex) {
            return badRequest();
        }
    }}
```

### 2.6.2)TeamController class

```
package controllers.team;

import javax.inject.Inject;

import play.mvc.*;
import play.libs.Json;

import services.team.TeamService;
import entities.team.FootballClub;

import java.util.List;
```



```
/**
* Class that handle FootballClub API data
*/
```

```
public class TeamController extends Controller {
        private final TeamService teamService;

        @Inject
        public TeamController(TeamService teamService){
                this.teamService = teamService;
        }

   /**
    * Get unsorted list of FootballClub teams registered in Premier League by
default.The result is {@code List<FootballClub>} or HTTP no content in case of no
results , if want to get the list sorted by wins,goals or points need to send  {@code
String orderBy} parameter in the request.
    * @param orderBy (default value = null) The string to get sorted list (points,wins or
goals)
    * @return {@code List<FootballClub>} or HTTP no content in case of no results
    */
   public Result getTeams(String orderBy) {
      try{
         List<FootballClub> footballClubList = (orderBy.isEmpty()) ?
teamService.getFootballLeague() : teamService.getFootballLeagueOrder(orderBy);
         if (footballClubList.size() > 0) {
            return ok(Json.toJson(footballClubList));
         } else {
            return noContent();
         }
      }catch(Exception ex) {
         return badRequest();
      }

   }

}
```

### 2.6.3)FrontendController class

```
package controllers;

import play.mvc.*;

import javax.inject.Inject;
import com.typesafe.config.Config;
import java.io.File;
```



```
/**
* Class that handle Angular UI app, act as a proxy that allow Play App to access all
```

```
resources or file in Angular UI Project
*/
public class FrontendController extends Controller {
   private final Config config;

   @Inject
   public FrontendController(Config config){
    this.config = config;
   }

   /**
   * handle access of UI index page
   */
   public Result index() {
    File file = new File("ui/src/index.html");
    return ok(file, true);
   }

   /**
   * Handle all UI Project resources so Play App can access it from its port
   */
   public Result assetOrDefault(String resource){
    if(resource.startsWith(config.getString("apiPrefix"))){
      return notFound();
    }else{
      if (resource.contains(".")){
        File file = new File("ui/src/"+resource);
        return ok(file, true);
      }else{
        return index();
      }
    }

   }

}
```

## 2.7) tasks

### 2.7.1) AppThread class

```
package tasks;

import cli.PremierLeagueApplication;
import javax.inject.Inject;
import com.typesafe.config.Config;
import java.io.IOException;

/**
* This class is responsible to handle thread that make CLI application to run in Play
Framework console.
* AppThread extends Java Thread which allows this process to run in background not in
foreground
*/
public class AppThread extends Thread{
        private final Config config;
        private final PremierLeagueApplication premierLeagueApp;

        /**
        * Constructor of AppThread
        * @param premierLeagueApp instance of PremierLeagueApplication class
        * @param config The Class which allows to access data from configuration file
like application.conf
        */
        @Inject
        public AppThread(PremierLeagueApplication premierLeagueApp, Config config){
                this.premierLeagueApp = premierLeagueApp;
                this.config = config;
        }

        /**
        * method that called when this class instantiated or loaded
        */
        @Override
   public void run(){
        System.out.println("Console App Running");

      String leagueFile = config.getString("file.league_file_name");
      String matchFile = config.getString("file.match_file_name");

      try{
         premierLeagueApp.runConsole(leagueFile, matchFile);
      }catch(IOException e){
         System.out.println(e.toString());
```

```
        }

    }

}
```

### 2.7.2) InitActorTask class

```
package tasks;

import entities.team.FootballClub;
import entities.match.Match;
import data.AppData;
import cli.PremierLeagueManager;

import java.util.List;
import javax.inject.Inject;
import com.typesafe.config.Config;

import play.DefaultApplication;

public class InitActorTask {
   private final Config config;
   private final AppThread appThread;
   private final PremierLeagueManager premierLeagueManager;

   private List<FootballClub> footballClubList = AppData.footballClubList;
   private List<Match> matchData = AppData.matchData;

   private final DefaultApplication defaultApp;

   /**
   * The constructor of InitActorTask Class
   * this method calls initialize method and start CLI application Thread
   */
   @Inject
   public InitActorTask(Config config, AppThread appThread, PremierLeagueManager
premierLeagueManager, DefaultApplication defaultApp){
      this.config = config;
      this.appThread = appThread;
      this.premierLeagueManager = premierLeagueManager;
      this.defaultApp = defaultApp;
      this.initialize();

      if(!defaultApp.isTest()){
         appThread.start();
      }
   }
```

```
    /**
    * this method load footballclub(league) data and match data from txt file
    */
    private void initialize(){
       System.out.println("Load Inital Data from File");

       String leagueFile = config.getString("file.league_file_name");
       String matchFile = config.getString("file.match_file_name");

       premierLeagueManager.retrieveStatisticsData(leagueFile);
       premierLeagueManager.retrieveMatchesData(matchFile);
    }
}
```

### 2.7.3) InitTasksModule class

```
package tasks;

import com.google.inject.AbstractModule;


/**
* module which allows to run command in startup of Play Framework
*/
public class InitTasksModule extends AbstractModule {

        /**
        * this function gets called when this class is loaded
        */
        @Override
        protected void configure() {
        bind(InitActorTask.class).asEagerSingleton();
        }

}
```

# 3) Unit Testing

## 3.1) TeamTest class

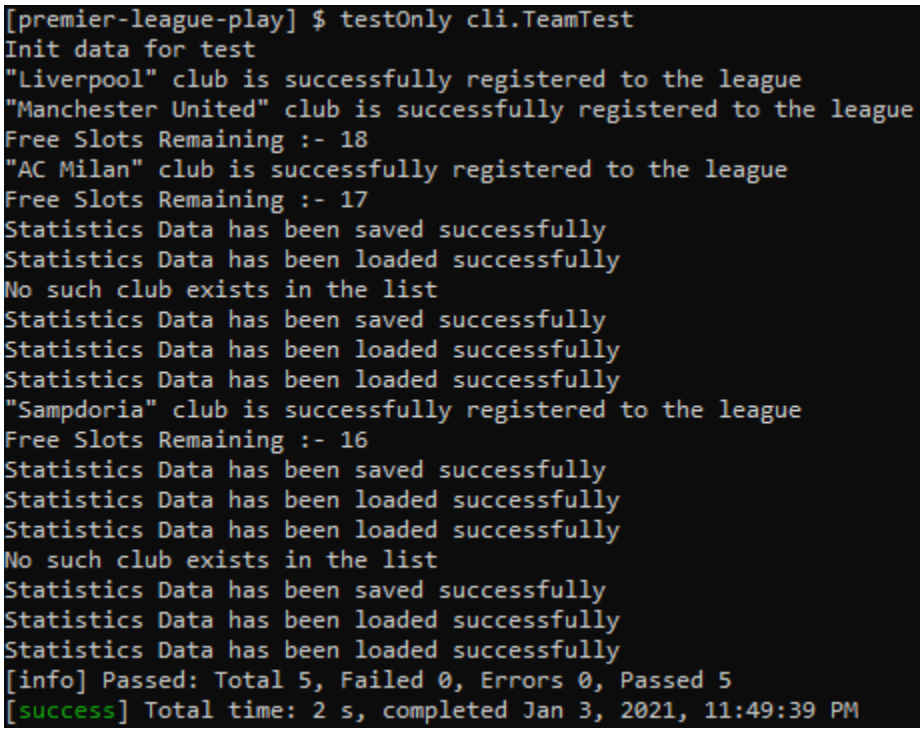
- This Class is used to ensure Team Feature of CLI part running well

```
package cli;

import cli.LeagueManager;
import cli.PremierLeagueManager;
import entities.team.FootballClub;
import org.junit.BeforeClass;
import org.junit.Test;

public class TeamTest {
   // this is to work with one instance of PremierLeagueManager
   private static LeagueManager premierLeague() {
      return new PremierLeagueManager();
   }

   /**
    * Execute initTestData before all test
    */

   @BeforeClass
   public static void initTeamTestData() {
      System.out.println("Init data for test");
      FootballClub liverpoolTeam = new FootballClub();
      liverpoolTeam.setClubName("Liverpool");
      liverpoolTeam.setClubLocation("England");
      liverpoolTeam.setClubManager("William");
      liverpoolTeam.setClubFormedYear(1978);

      FootballClub manchesterTeam = new FootballClub();
      manchesterTeam.setClubName("Manchester United");
      manchesterTeam.setClubLocation("England");
      manchesterTeam.setClubManager("Ben");
      manchesterTeam.setClubFormedYear(1978);

      FootballClub milanTeam = new FootballClub();
      milanTeam.setClubName("AC Milan");
      milanTeam.setClubLocation("Italy");
      milanTeam.setClubManager("Carletto");
      milanTeam.setClubFormedYear(1899);
```

```
        premierLeague().addFootballClub(liverpoolTeam);
        premierLeague().addFootballClub(manchesterTeam);
        premierLeague().addFootballClub(milanTeam);

        premierLeague().saveStatisticsData("LeagueFileTest.txt");
    }

    /**
    * add football club test
    */
    @Test
    public void addFootballClubTest() {
        boolean findNewClub = false;
        FootballClub footballClub = new FootballClub();
        footballClub.setClubName("Sampdoria");
        footballClub.setClubLocation("Italy");
        footballClub.setClubManager("Genaro Gatuso");
        footballClub.setClubFormedYear(1878);

        premierLeague().addFootballClub(footballClub);  // add footballClub to array list

        premierLeague().saveStatisticsData("LeagueFileTest.txt");  // save all data in file

        premierLeague().retrieveStatisticsData("LeagueFileTest.txt");  // retrieve all data to
check if the object really saves in it

        // Loop the array list (getClubs return the football array list from premier League)
        for (FootballClub footballClub1 : premierLeague().getClubs()) {
            if (footballClub1.getClubName().equals(footballClub.getClubName())) {
                findNewClub = true;
                break;
            }
        }
        assert findNewClub;
    }

    /**
    * Delete test
    */
    @Test
    public void deleteExistedFootballClubTest() {
        boolean deleteClub = true;
        premierLeague().retrieveStatisticsData("LeagueFileTest.txt");
        premierLeague().deleteClub("Sampdoria");
        premierLeague().saveStatisticsData("LeagueFileTest.txt");
        premierLeague().retrieveStatisticsData("LeagueFileTest.txt");
        for (FootballClub footballClub : premierLeague().getClubs()) {
            if (footballClub.getClubName().equals("Sampdoria")) {
```

```
            deleteClub = false;
            break;
        }
      }
      assert deleteClub;
   }

   @Test
   public void deleteNotExistedFootballClubTest() {
      boolean deleteClub = false;
      premierLeague().retrieveStatisticsData("LeagueFileTest.txt");
      premierLeague().deleteClub("Unknown Team");
      premierLeague().saveStatisticsData("LeagueFileTest.txt");
      premierLeague().retrieveStatisticsData("LeagueFileTest.txt");

      for (FootballClub footballClub : premierLeague().getClubs()) {
         if (footballClub.getClubName().equals("Unknown Team")) {
            deleteClub = true;
            break;
         }
      }
      assert !deleteClub;
   }

   /**
   * Search Test
   */
   @Test
   public void findTeamStatisticsByNameTest() {
      premierLeague().retrieveStatisticsData("LeagueFileTest.txt");
      FootballClub footballClubTest = premierLeague().findTeamStatisticsByName("AC
Milan");
      assert footballClubTest != null;
   }

   /**
   * Test not existed team by name
   */

   @Test
   public void findNotExistedTeamStatisticsByNameTest() {
      premierLeague().retrieveStatisticsData("LeagueFileTest.txt");
      FootballClub footballClubTest =
premierLeague().findTeamStatisticsByName("Unknown Team");
      assert footballClubTest == null;
   }

}
```

Screenshot of TeamTest class




## 3.2) TeamControllerTest class

- This Class is used to ensure Team Feature of GUI part running well

```
package controllers;

import org.junit.Test;
import play.Application;
import play.inject.guice.GuiceApplicationBuilder;
import play.mvc.Http;
import play.mvc.Result;
import play.test.WithApplication;
```

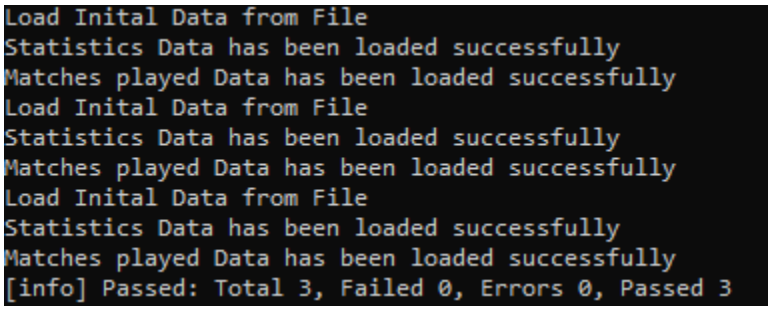
```
import static org.junit.Assert.assertEquals;
import static play.mvc.Http.Status.OK;
import static play.test.Helpers.GET;
import static play.test.Helpers.route;

/**
* This class ensure TeamController running well
* Referred :-
https://www.playframework.com/documentation/2.8.x/JavaFunctionalTest
*/
public class TeamControllerTest extends WithApplication {


  /**
  * Override play app configuration for the test
  */
  @Override
  protected Application provideApplication() {
    return new GuiceApplicationBuilder()
 .configure("file.league_file_name", "LeagueFileTest.txt")
    .configure("file.match_file_name", "MatchFileTest.txt")
 .build();
  }

  /**
  * Test team list order by points, should be http 200 response code
  */
  @Test
  public void getTeamsOrderByPoints() {
    Http.RequestBuilder request = new Http.RequestBuilder()
        .method(GET)
        .uri("http://localhost:9000/premierleague/team/teams?orderBy=points");

    Result result = route(app, request);
    assertEquals(OK, result.status());
  }

  /**
  * Test team list order by goals, should be http 200 response code
  */
      @Test
  public void getTeamsOrderByGoals() {
    Http.RequestBuilder request = new Http.RequestBuilder()
        .method(GET)
        .uri("http://localhost:9000/premierleague/team/teams?orderBy=goals");
```



```
    Result result = route(app, request);
    assertEquals(OK, result.status());
```

```
    }

    /**
    * Test team list order by wins, should be http 200 response code
    */
          @Test
    public void getTeamsOrderByWins() {
       Http.RequestBuilder request = new Http.RequestBuilder()
             .method(GET)
             .uri("http://localhost:9000/premierleague/team/teams?orderBy=wins");

       Result result = route(app, request);
       assertEquals(OK, result.status());
    }
}
```

## Screenshot of TeamControllerTest class




### 3.3) MatchTest class

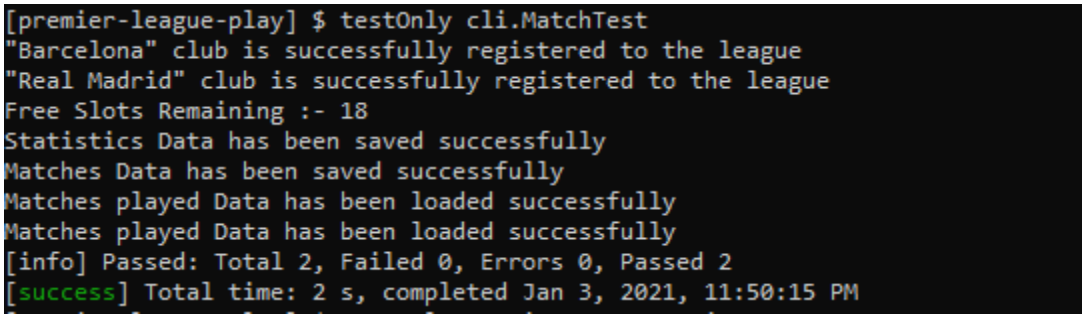
- This Class is used to ensure Match Feature of CLI part running well

```
package cli;

import cli.LeagueManager;
import cli.PremierLeagueManager;
import entities.match.Match;
import entities.team.FootballClub;
import org.junit.BeforeClass;
import org.junit.Test;

/**
```

```
* This Class is used to ensure Match Feature running well
*/
public class MatchTest {
   /**
    * this is to work with one instance of PremierLeagueManager
    */
   private static LeagueManager premierLeague() {
      return new PremierLeagueManager();
   }

   /**
    * Execute initTestData before all test
    */
   @BeforeClass
   public static void initMatchTestData() {
      FootballClub homeClub = new FootballClub();
      homeClub.setClubName("Barcelona");
      homeClub.setClubLocation("Spain");
      homeClub.setClubManager("John");
      homeClub.setClubFormedYear(1899);

      FootballClub awayClub = new FootballClub();
      awayClub.setClubName("Real Madrid");
      awayClub.setClubLocation("Spain");
      awayClub.setClubManager("Peter");
      awayClub.setClubFormedYear(1899);

      premierLeague().addFootballClub(homeClub);
      premierLeague().addFootballClub(awayClub);
      premierLeague().saveStatisticsData("LeagueFileTest.txt");

      premierLeague().footBallMatch(homeClub.getClubName(), 1,
awayClub.getClubName(), 2, "11/12/2020");
      premierLeague().footBallMatch(homeClub.getClubName(), 3,
awayClub.getClubName(), 4, "10/12/2020");
      premierLeague().footBallMatch(homeClub.getClubName(), 2,
awayClub.getClubName(), 1, "28/12/2020");

      premierLeague().saveMatchesData("MatchFileTest.txt");
   }

   @Test
   public void footballMatchDatePlayedTest() {
      premierLeague().retrieveMatchesData("MatchFileTest.txt");
      boolean findMatch = false;
      for (Match match : premierLeague().getMatchData()) {
         if (match.getDatePlayed().equals("28/12/2020")) {
            findMatch = true;
```

```
            break;
          }
        }
        assert findMatch;
    }


    /**
    * test if match not exist
    */
    @Test
    public void footballMatchDatePlayedNotExistTest() {
        premierLeague().retrieveMatchesData("MatchFileTest.txt");
        boolean findMatch = false;
        for (Match match : premierLeague().getMatchData()) {
            if (match.getDatePlayed().equals("10/10/2020")) {
                findMatch = true;
                break;
            }
        }
        assert !findMatch;
    }
}
```

## Screenshot of MatchTest class



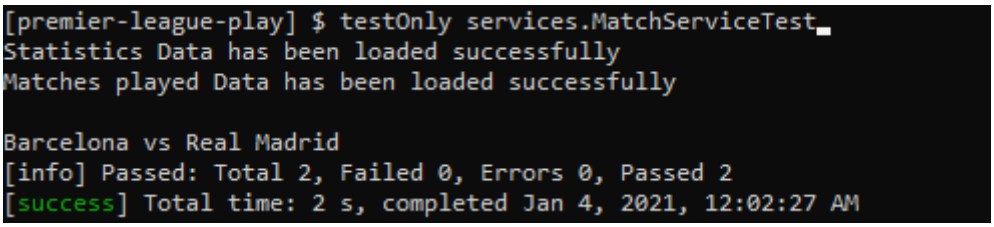
### 3.4) MatchServiceTest class

```
package services;

import org.junit.BeforeClass;
import org.junit.Test;
import org.junit.Assert;

import java.io.File;
import java.util.Date;
import java.util.List;
import java.text.SimpleDateFormat;
import java.text.ParseException;

import entities.match.Match;
import entities.team.FootballClub;
import services.match.*;
import services.team.*;
import cli.PremierLeagueManager;

import com.typesafe.config.Config;
import com.typesafe.config.ConfigFactory;


/**
* This class ensure MatchService Class running well.
* MatchService.class is usefull to deliver match data to controller
*/
public class MatchServiceTest {
	private static MatchService matchService;
	private static TeamService teamService;
	private static PremierLeagueManager premierLeagueManager;


	/**
	* Initial data & object before run test
	*/
	@BeforeClass
	public static void initializeTest(){
		premierLeagueManager = new PremierLeagueManager();
		premierLeagueManager.retrieveStatisticsData("LeagueFileTest.txt");
		premierLeagueManager.retrieveMatchesData("MatchFileTest.txt");

		String configFile = "conf/application_test.conf";
	Config config = ConfigFactory.parseFile(new File(configFile));
	teamService = new TeamServiceImpl(config);
	matchService = new MatchServiceImpl(config, teamService);
	}
```

```
	/**
	* Test randomMatch() method ensure it returns Match data
	*/
	@Test
	public void generateRandomMatchTest(){
		boolean findClub = false;
		Match randomMatch = matchService.randomMatch();

		for (FootballClub footballClub: teamService.getFootballLeague()) {

	if(footballClub.getClubName().equalsIgnoreCase(randomMatch.getHomeTeam()
)){
				findClub = true;
			}
		}

		System.out.println();
		System.out.println(randomMatch.getHomeTeam()+" vs
"+randomMatch.getAwayTeam());
		Assert.assertEquals(true, findClub);
	}


	/**
	* This test ensure match data sorted correctly
	*/
	@Test
	public void sortedMatchDataTest(){
		boolean sorted = false;
		List<Match> sortedMatchs = matchService.getMatchesByDateAsc();
		int totalMatch = sortedMatchs.size();

		Match firstMatch = sortedMatchs.get(0);
		Match lastMatch = sortedMatchs.get(totalMatch - 1);

		SimpleDateFormat format = new SimpleDateFormat("dd/MM/yyyy");

		try{
			Date firstMatchDate = format.parse(firstMatch.getDatePlayed());
			Date lastMatchDate = format.parse(lastMatch.getDatePlayed());
			if (firstMatchDate.before(lastMatchDate)) {
				sorted = true;
			}
		}catch (ParseException pe) {
			System.out.println(pe.toString());
		}catch (Exception e){
			System.out.println(e.toString());
		}
```

```
                Assert.assertEquals(true, sorted);
        }

}
```

Screenshot of MatchServiceTest class




### 3.5) MatchControllerTest class

- This Class is used to ensure Match Feature of GUI part running well

```
package controllers;

import org.junit.Test;
import play.Application;
import play.inject.guice.GuiceApplicationBuilder;
import play.mvc.Http;
import play.mvc.Result;
import play.test.WithApplication;

import static org.junit.Assert.assertEquals;
import static play.mvc.Http.Status.OK;
import static play.test.Helpers.GET;
import static play.test.Helpers.route;

/**
* This class ensure MatchController running well
* Referred :-
https://www.playframework.com/documentation/2.8.x/JavaFunctionalTest
*/
public class MatchControllerTest extends WithApplication {

   /**
```

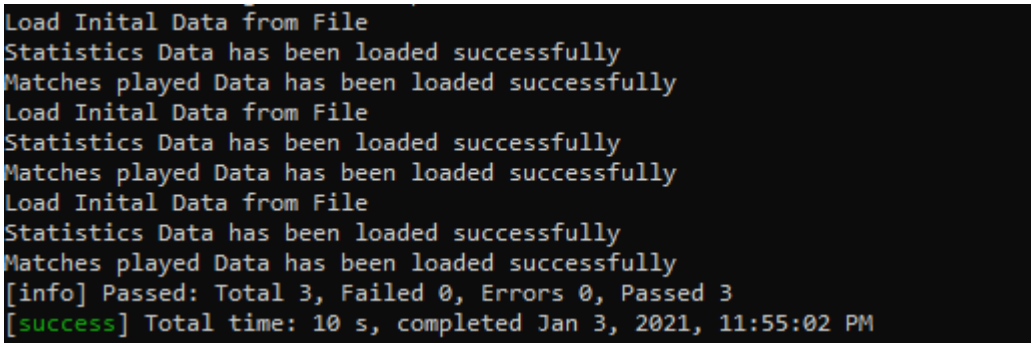
```
  * Override play app configuration for the test
  */
  @Override
  protected Application provideApplication() {
     return new GuiceApplicationBuilder()
 .configure("file.league_file_name", "LeagueFileTest.txt")
     .configure("file.match_file_name", "MatchFileTest.txt")
 .build();
  }

  /**
  * search match by date API Test , should be http 200 response code
  */
  @Test
  public void getMatchesByDateTest() {
     Http.RequestBuilder request = new Http.RequestBuilder()
           .method(GET)

.uri("http://localhost:9000/premierleague/match/matches?date=28/12/2020");

     Result result = route(app, request);
     assertEquals(OK, result.status());
  }

  /**
  * match list order by date ascending API Test , should be http 200 response code
  */
        @Test
  public void getMatchesOrderByDateTest() {
     Http.RequestBuilder request = new Http.RequestBuilder()
           .method(GET)
           .uri("http://localhost:9000/premierleague/match/matches/order-by-date");

     Result result = route(app, request);
     assertEquals(OK, result.status());
  }

  /**
  * random match API Test , should be http 200 response code
  */
        @Test
  public void getRandomMatchTest() {
     Http.RequestBuilder request = new Http.RequestBuilder()
           .method(GET)
           .uri("http://localhost:9000/premierleague/match/random");
```



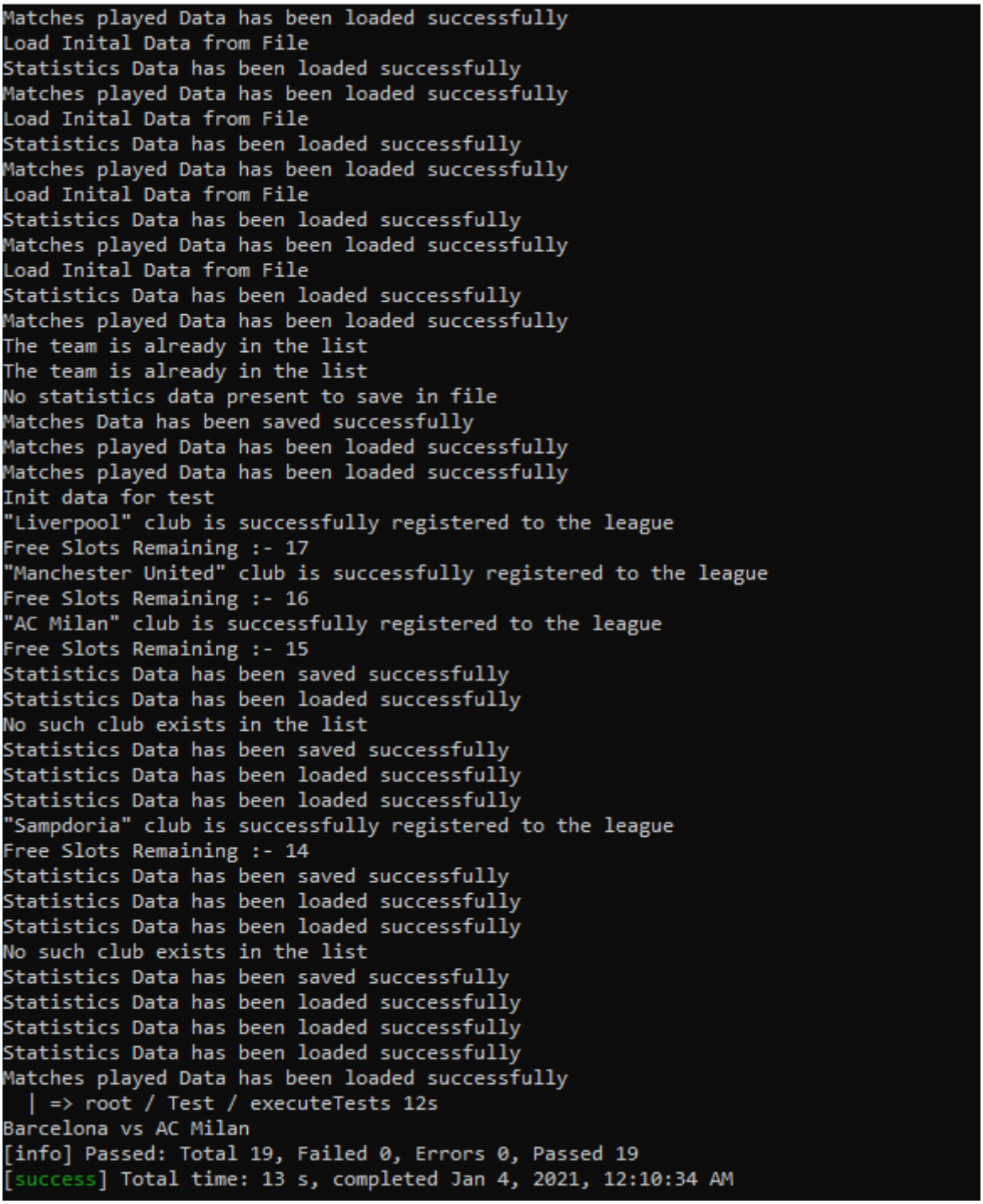
```
     Result result = route(app, request);
     assertEquals(OK, result.status());
```

```
    }
}
```

## Screenshot of MatchControllerTest class



Screenshot when all classes are run



# 4) UI Components - Angular

## 4.1)Main page (main-gui)

### 4.1.1) main-gui.component.html

- This consists the main page of GUI.

```
<div class="mainGui">
<h1>Welcome to Premier League Application Managament</h1>
</div>
```

### 4.1.2) main-gui.component.ts

```
import { Component, OnInit } from '@angular/core';

@Component({
 selector: 'app-main-gui',
 templateUrl: './main-gui.component.html',
 styleUrls: ['./main-gui.component.scss']
})
export class MainGuiComponent implements OnInit {
 constructor() { }

 ngOnInit() {
 }

}
```

## 4.2)Navigation Bar

### 4.2.1)app.component.html

```
<header class="header-fixed">
        <div class="header-limiter">
         <span class="toggle-btn" style="float:left; padding:6px; padding-right:16px;"
(click)="toogleSideBar()"  >
                <mat-icon *ngIf="!toogle; else close"
style="cursor:pointer">reorder</mat-icon>
         </span>
                 <a href="#"><img src="../assets/images/logo.png" alt="logo" /> </a>
        </div>
        <ng-template #close>
```

```
  <mat-icon style="cursor:pointer">close</mat-icon>
</ng-template>
</header>

<!-- to prevent the content of the page from jumping up -->
<div class="header-fixed-placeholder"></div>
<div id="sidebar" [class.active]="toogle">
  <div class="list">
    <!-- navigation bar options -->
    <div class="item"  [routerLink]="['/main-gui']" >- Home</div>
    <div class="item" (click)="toogleSideBar()" [routerLink]="['/first-gui']" >- VIEW
PREMIER LEAGUE TABLE</div>
    <div class="item" (click)="toogleSideBar()"[routerLink]="['/second-gui']">- PLAY A
RANDOM MATCH</div>
    <div class="item" (click)="toogleSideBar()" [routerLink]="['/third-gui']">- SEARCH
MATCHES PLAYED BY DATE</div>
    <div class="item" (click)="toogleSideBar()" [routerLink]="['/fourth-gui']">- VIEW AND
SORT MATCHES PLAYED BY DATE</div>
  </div>
</div>
<div class="place">
<router-outlet></router-outlet>
</div>
```

### 4.2.2) app.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss']
})
export class AppComponent {
  title = 'premier-league';
  toogle = true;

  //show or hide the navigation bar
  toogleSideBar():void{
            this.toogle = !this.toogle
  }
}
```

### 4.2.3) app-routing.module.ts

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';
import { ErrorComponent } from './error/error.component';
import { FirstGuiComponent } from './first-gui/first-gui.component';
import { FourthGuiComponent } from './fourth-gui/fourth-gui.component';
import { MainGuiComponent } from './main-gui/main-gui.component';
import { SecondGuiComponent } from './second-gui/second-gui.component';
import { ThirdGuiComponent } from './third-gui/third-gui.component';

//routes for Angular Premier League App
const routes: Routes = [
 {
  path: '',
  redirectTo: 'main-gui',
  pathMatch: 'full'
 },
 {
  path: 'main-gui',
  component: MainGuiComponent
 },
 {
  path: 'first-gui',
  component: FirstGuiComponent
 },
 {
  path: 'second-gui',
  component: SecondGuiComponent
 },
 {
  path: 'third-gui',
  component: ThirdGuiComponent
 },
 {
  path: 'fourth-gui',
  component: FourthGuiComponent
 },
 {
  path: '**',
  component: ErrorComponent
 }
];

@NgModule({
 imports: [RouterModule.forRoot(routes)],
 exports: [RouterModule]
})
```

```
export class AppRoutingModule { }
```

## 4.3) CSS and Interfaces

### 4.3.1) index.ts

```
//types for angular premier league app
export interface IScore {
   clubName: string;
   totalMatches: number;
   wins: number;
   draws: number;
   defeats: number;
   points: number;
   goalsScored: number;
   goalsReceived: number;
   goalsDifference: number;
}

export interface IClub {
   date: Date;
   home_club: string;
   home_score: number;
   away_score: number;
   away_club: string;
}

export interface IFootballClub {
   wins: number;
   draws: number;
   defeats: number;
   goalsScored: number;
   goalsReceived: number;
  goalsDifference: number;
   totalMatches: number;
   points: number;

   clubName: string;
   clubLocation: string;
   clubManager: string;
   clubFormedYear: number;
}

export interface IMatch {
   datePlayed: string;
   awayScore: number;
   awayTeam: IFootballClub;
```

```
    homeScore: number;
    homeTeam: IFootballClub;
}
```

### 4.3.2) index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Premier League</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="assets/images/favicon.ico">
  <link
href="https://fonts.googleapis.com/css?family=Roboto:300,400,500&display=swap"
rel="stylesheet">
  <link rel="stylesheet"
        href="https://fonts.googleapis.com/css?family=Langar">
  <link href="https://fonts.googleapis.com/icon?family=Material+Icons"
rel="stylesheet">
  <style>
    body {
      background-image: url(assets/images/2017.jpg);
      background-repeat: round;
            color: white !important;
    }

    .table td {
      text-align: center;
            border-top: 1px solid #858b92 !important;
    }

          .table-striped tbody tr:nth-of-type(odd) {
    background-color: rgb(15 90 162) !important;
    color: white;
    font-weight: bold;
}

          .table-striped tbody tr:nth-of-type(even) {
    background-color: rgb(90 158 224 / 71%) !important;
    color: white;
    font-weight: bold;
}

          .table th {
          background: #104375;
          border-top: 4px solid #346492 !important;
          border-bottom: 4px solid #346492 !important;
```

```
        border-left: 4px solid #346492 !important;
        border-right: 4px solid #346492 !important;
        color: white;

        }

 .table thead th {
   text-align: center;
   font-size: 21px;
 }

 .container {
   text-align: center;
 }

 .score-card-list {
   display: flex;
   flex-direction: column;
   align-items: center;
 }

 .score-card {
   width: 550px;
   display: flex !important;
 }

 .home {
   flex: 1;
   text-align: center;
 }

 .away {
   flex: 1;
   text-align: center;
 }

 .score {
   width: 100px;
 }

        #sidebar {
position:fixed;
top: 12%;
left:-200px;
width:200px;
height:100%;
background:#104375;
transition:all 300ms linear;
```

```
  z-index: 3;
}
#sidebar.active {
  left:0px;
  z-index:3
}
.place{
    height: auto;
    transition: all 300ms ease-in-out;
}
#sidebar .toggle-btn {
  position:absolute;
  left:220px;
  top:10px;
  z-index: 3;
  cursor:pointer;
}
#sidebar .toggle-btn span {
  display:block;
  width:30px;
  height:5px;
  background:#151719;
  margin:5px 0px;
  cursor:pointer;
}
#sidebar div.list div.item {
  padding:15px 10px;
  color:#fcfcfc;
  text-transform:uppercase;
  font-size:15px;
  cursor:pointer;
  border: none;
  border-bottom: 1px solid #20568b;
}

#sidebar div.list div.item:hover {
 color: orange
}

#sidebar div.list div.item:focus {
 background-color: grey
}

.header-fixed {
        background-color:#104375;
        padding: 20px 40px;
        height: 80px;
        color: #ffffff;
```

```
	box-sizing: border-box;
	top:-100px;

	-webkit-transition:top 0.3s;
	transition:top 0.3s;
}

.header-fixed .header-limiter {
	max-width: 1200px;
	margin: 0 auto;
}

/*	The header placeholder. It is displayed when the header is fixed to the top of
the
	browser window, in order to prevent the content of the page from jumping up.
*/

.header-fixed-placeholder{
	height: 80px;
	display: none;
}

/* Logo */

.header-fixed .header-limiter h1 {
	float: left;
	font: normal 28px Cookie, Arial, Helvetica, sans-serif;
	line-height: 40px;
	margin: 0;
}

h2{
	line-height: 40px;
	font: normal 28px Langar;
	font-size: 3rem !important;
	text-align: center;
}

.header-fixed .header-limiter h1 span {
	color: #5383d3;
}

/* The navigation links */

.header-fixed .header-limiter a {
	color: #ffffff;
	text-decoration: none;
}
```

```
.header-fixed .header-limiter nav {
	font:16px Arial, Helvetica, sans-serif;
	line-height: 40px;
	float: right;
}

.header-fixed .header-limiter nav a{
	display: inline-block;
	padding: 0 5px;
	text-decoration:none;
	color: #ffffff;
	opacity: 0.9;
}

.header-fixed .header-limiter nav a:hover{
	opacity: 1;
}

.header-fixed .header-limiter nav a.selected {
	color: #608bd2;
	pointer-events: none;
	opacity: 1;
}

/* Fixed version of the header */

body.fixed .header-fixed {
	padding: 10px 40px;
	height: 50px;
	position: fixed;
	width: 100%;
	top: 0;
	left: 0;
	z-index: 1;
}

body.fixed .header-fixed-placeholder {
	display: block;
}

body.fixed .header-fixed .header-limiter h1 {
	font-size: 24px;
	line-height: 30px;
}
```



```
body.fixed .header-fixed .header-limiter nav {
	line-height: 28px;
```

```
        font-size: 13px;
}


/* Making the header responsive */

@media all and (max-width: 600px) {

        .header-fixed {
                padding: 20px 0;
                height: 75px;
        }

        .header-fixed .header-limiter h1 {
                float: none;
                margin: -8px 0 10px;
                text-align: center;
                font-size: 24px;
                line-height: 1;
        }

        .header-fixed .header-limiter nav {
                line-height: 1;
                float:none;
        }

        .header-fixed .header-limiter nav a {
                font-size: 13px;
        }

        body.fixed .header-fixed {
                display: none;
        }

}

.legend{
        border: 4px solid #346492;
   font-weight: bold;
   padding: 20px;
   text-align: initial;
   background-color: #104375;
}

.RedButton {
        box-shadow: 3px 4px 0px 0px #8a2a21;
        background-color:#c62d1f;
        border-radius:18px;
```

```
	border:1px solid #d02718;
	display:inline-block;
	cursor:pointer;
	color:#ffffff;
	font-family:Arial;
	font-size:17px;
	padding:7px 25px;
	text-decoration:none;
	text-shadow:0px 1px 0px #810e05;
}
.RedButton:hover {
	background-color:#f24437;
}
.RedButton:active {
	position:relative;
	top:1px;
}

.BlackButton {
	box-shadow: 3px 4px 0px 0px black;
	background-color:black;
	border-radius:18px;
	border:1px solid black;
	display:inline-block;
	cursor:pointer;
	color:#ffffff;
	font-family:Arial;
	font-size:17px;
	padding:7px 25px;
	text-decoration:none;
	text-shadow:0px 1px 0px #810e05;
}
.BlackButton:hover {
	background-color:gray;
}
.BlackButton:active {
	position:relative;
	top:1px;
}

.GreenButton {
	box-shadow: 3px 4px 0px 0px green;
	background-color:#20c997;
	border-radius:18px;
	border:1px solid green;
	display:inline-block;
	cursor:pointer;
	color:#ffffff;
```

```
	font-family:Arial;
	font-size:17px;
	padding:7px 25px;
	text-decoration:none;
	text-shadow:0px 1px 0px #810e05;
}
.GreenButton:hover {
	background-color:#7ead7e;
}
.GreenButton:active {
	position:relative;
	top:1px;
}

.BlueButton {
	box-shadow: 3px 4px 0px 0px #104375;
	background-color:#104375;
	border-radius:18px;
	border:1px solid #104375;
	display:inline-block;
	cursor:pointer;
	color:#ffffff;
	font-family:Arial;
	font-size:17px;
	padding:5px;
	text-decoration:none;
	text-shadow:0px 1px 0px #810e05;
}
.BlueButton:hover {
	background-color:#104375;
}
.BlueButton:active {
	position:relative;
	top:1px;
}

.mainGui{
}

h1 {
  -webkit-transition: all 3s ease-out;
       transition: all 3s ease-out;
  -webkit-animation: pulsate 8s ease-out infinite;
       animation: pulsate 8s ease-out infinite;
  position: absolute;
  top: 50%;
  left: 50%;
  -webkit-transform: translate(-50%, -50%);
```

```
        transform: translate(-50%, -50%);
  margin: 0;
  font-family: Helvetica, Arial, Sand-serif;
  font-weight: bold !important;
  font-size: 4em !important;
  color: white;
  text-align: center;
  text-transform: uppercase; }

h1:hover {
  color: white; }

/*** Keyframes - used with the "animation" property in the h1 ***/
@-webkit-keyframes pulsate {
  0%, 20% {
    -webkit-transform: translate(-50%, -50%) scale(1);
          transform: translate(-50%, -50%) scale(1);
    -webkit-filter: blur(5px);
          filter: blur(5px); }
  50%, 70% {
    -webkit-transform: translate(-50%, -50%) scale(1.5);
          transform: translate(-50%, -50%) scale(1.5);
    -webkit-filter: none;
          filter: none; }
  100% {
    -webkit-transform: translate(-50%, -50%) scale(1);
          transform: translate(-50%, -50%) scale(1);
    -webkit-filter: blur(5px);
          filter: blur(5px); } }
@keyframes pulsate {
  0%, 20% {
    -webkit-transform: translate(-50%, -50%) scale(1);
          transform: translate(-50%, -50%) scale(1);
    -webkit-filter: blur(5px);
          filter: blur(5px); }
  50%, 70% {
    -webkit-transform: translate(-50%, -50%) scale(1.5);
          transform: translate(-50%, -50%) scale(1.5);
    -webkit-filter: none;
          filter: none; }
  100% {
    -webkit-transform: translate(-50%, -50%) scale(1);
          transform: translate(-50%, -50%) scale(1);
    -webkit-filter: blur(5px);
          filter: blur(5px); } }
```



```
/*** Media queries for adjusting to different screen sizes ***/
@media only screen and (max-width: 600px) {
```

```
 h1 {
   font-size: 2em; }
}

#sidebar.active ~ .place {
   padding-left: 14%;
          transition:all 300ms linear;
}
#sidebar {
   padding:1%;
          transition:all 300ms linear;
}

.header-limiter img{
          float: left;
   width: auto;
   height: 71px;
   margin-top: -17px;
}

 </style>
</head>

<body>
 <app-root></app-root>
</body>

</html>
```

## 4.4) View Premier League Table (first-gui)

### 4.4.1) first-gui.component.html

1. Consists of Premier League statistics table which can be sorted according to Number of Wins,Number of Points and Number of Goals Scored.

```
<div class="container mt10">
 <div class="row">
   <div class="col-lg-9">
          <h2>Premier League Table</h2>
     <div class="table-responsive">
       <table class="table table-striped">
```

```
<!-- table headings -->
<thead>
        <tr>
         <th>#</th>
          <th>Club</th>
          <th>MP</th>
          <th>W</th>
          <th>D</th>
          <th>L</th>
          <th>GF</th>
          <th>GA</th>
          <th>GD</th>
          <th>Pts</th>

        </tr>
       </thead>
       <tbody>
         <!-- table data -->
        <tr *ngFor="let score of scores; let i = index">
          <td>{{i + 1}}</td>
          <td>{{ score.clubName }}</td>
          <td> {{ score.totalMatches }} </td>
          <td> {{ score.wins }} </td>
          <td> {{ score.draws }} </td>
          <td> {{ score.defeats }} </td>
          <td> {{ score.goalsScored }} </td>
          <td> {{ score.goalsReceived }} </td>
          <td> {{ score.goalsDifference }} </td>
           <td> {{ score.points }} </td>
        </tr>
       </tbody>
     </table>
   </div>
         <div class="row mt10">
                <div class="col-lg-12">
                 <div class="float-right">
                        <button class="RedButton mx10" (click)="sort('wins')">Sort By
Wins</button>
                        <button class="BlackButton mx10" (click)="sort('points')">Sort By
Points</button>
                        <button class="GreenButton mx10" (click)="sort('goals')">Sort By
Goals Scored</button>
                  </div>
                </div>
          </div>
  </div>
        <!-- legend displayed on right -->
        <div class="col-lg-3">
```

```
                <div class="legend">
                        <p>MP - Matches Played</p>
                        <p>W - Wins</p>
                        <p>D - Draws</p>
                        <p>L - Loss (Defeat)</p>
                        <p>GF - Goals Scored</p>
                        <p>GA - Goals Received</p>
                        <p>GD - Goals Difference</p>
                        <p>Pts - Points</p>

                </div>
        </div>
 </div>

</div>
```

### 4.4.2) first-gui.component.ts

```
import { Component, OnInit } from '@angular/core';
import { IMatch } from '../models';
import { DataService } from '../services/data.service';

@Component({
 selector: 'app-fourth-gui',
 templateUrl: './fourth-gui.component.html',
 styleUrls: ['./fourth-gui.component.scss']
})
export class FourthGuiComponent implements OnInit {

 matches : IMatch[] = [];

 constructor(private dataService: DataService) {
 }

 // Sending request to get teams
 ngOnInit() {
  this.dataService.sendGetRequest('match/matches').subscribe((data: IMatch[]) => {
   this.matches = data;
  });
 }

 // Sending request to get teams by order to display data in table
 sort() {
  this.dataService.sendGetRequest('match/matches/order-by-date').subscribe((data:
IMatch[]) => {
   this.matches = data;
  });
 }
}
```

## 4.5) Play A Random Match (second-gui)

### 4.5.1) second-gui.component.html

```
<div class="container mt10">
   <div class="row">
      <div class="col-lg-9">
                  <h2>Matches Played</h2>
         <div class="float-right py10">
            <button class="GreenButton" (click)="randomMatch()">Play A Random
Match</button>
         </div>
         <div class="table-responsive">
            <table class="table table-striped">
               <!-- table headings -->
               <thead>
                  <tr>
                     <th>#</th>
                     <th>Club</th>
                     <th>MP</th>
                     <th>W</th>
                     <th>D</th>
                     <th>L</th>
                     <th>GF</th>
                     <th>GA</th>
                     <th>GD</th>
                     <th>Pts</th>
                  </tr>
               </thead>
               <tbody>
                  <!-- table data -->
                  <tr *ngFor="let score of scores; let i = index">
                     <td>{{i + 1}}</td>
                     <td>{{ score.clubName }}</td>
                     <td> {{ score.totalMatches }} </td>
                     <td> {{ score.wins }} </td>
                     <td> {{ score.draws }} </td>
                     <td> {{ score.defeats }} </td>
                     <td> {{ score.goalsScored }} </td>
                     <td> {{ score.goalsReceived }} </td>
                     <td> {{ score.goalsDifference }} </td>
                     <td> {{ score.points }} </td>
                  </tr>
               </tbody>

            </table>
```

```
        </div>
      </div>
                <!-- legend displayed on right -->
                <div class="col-lg-3">
                <div class="legend">
                        <p>MP - Matches Played</p>
                        <p>W - Wins</p>
                        <p>D - Draws</p>
                        <p>L - Loss (Defeat)</p>
                        <p>GF - Goals Scored</p>
                        <p>GA - Goals Received</p>
                        <p>GD - Goals Difference</p>
<p>Pts - Points</p>
                </div>
          </div>
    </div>
</div>
```

### 4.5.2) second-gui.component.ts

```
import { Component, OnInit } from '@angular/core';
import { MatDialog } from '@angular/material';
import { IMatch, IScore } from '../models';
import { SecondGuiModalComponent } from '../second-gui-modal/second-gui-
modal.component';
import { DataService } from '../services/data.service';
@Component({
  selector: 'app-second-gui',
  templateUrl: './second-gui.component.html',
  styleUrls: ['./second-gui.component.scss']
})
export class SecondGuiComponent implements OnInit {

  scores: IScore[] = [];

  constructor(
    public dialog: MatDialog, private dataService: DataService
  ) { }

  ngOnInit() {
    this.getTeams();
  }

  // Sending request to get teams order by points for display in data table
  private getTeams(): void {
    this.dataService.sendGetRequest('team/teams', { orderBy: 'points' }).subscribe((data:
IScore[]) => {
      this.scores = data;
```

```
  });
 }

 // Sending request to get random method to display in Mat Dialog
 randomMatch() {
  this.dataService.sendGetRequest('match/random').subscribe((data: IMatch) => {
   this.dialog.open(SecondGuiModalComponent, {
    data
   }).afterClosed().subscribe((evt: any) => {
    this.getTeams();
   });
  });
 }
}
```

### 4.5.3) second-gui-modal.component.html

```
<div class="container">
                <button mat-icon-button mat-dialog-close aria-label="close"
style="float:right; margin-top:-21px;">
                        <mat-icon>close</mat-icon>
                </button>
                 <h5 style="text-align: initial">Date: {{data.datePlayed}}</h5>
    <div class="score-card">
      <h3 class="home">{{data.homeTeam}}</h3>
      <h3 class="score"></h3>
      <h3 class="away">{{data.awayTeam}}</h3>
    </div>
                <div class="score-card">
                   <h3 class="home"></h3>
      <h3 class="score">vs</h3>
      <h3 class="away"></h3>
    </div>

    <div class="score-card">
      <h3 class="home">{{data.homeScore}}</h3>
      <h3 class="score"></h3>
      <h3 class="away">{{data.awayScore}}</h3>
    </div>

</div>
```

### 4.5.4) second-gui-modal.component.ts

```
import { Component, Inject, OnInit } from '@angular/core';
import { MatDialogRef, MAT_DIALOG_DATA } from '@angular/material';
```

```
@Component({
 selector: 'app-second-gui-modal',
 templateUrl: './second-gui-modal.component.html',
 styleUrls: ['./second-gui-modal.component.scss']
})
export class SecondGuiModalComponent implements OnInit {

 // Configuration for Mat Dialog Component
 constructor(
  public dialogRef: MatDialogRef<SecondGuiModalComponent>,
  @Inject(MAT_DIALOG_DATA) public data: SecondGuiModalComponent
 ) { }

 ngOnInit() {
 }

}
```

## 4.6) SEARCH MATCHES PLAYED WITH DATE (third-gui)

### 4.6.1) third-gui.component.html

```
<div class="container mt10">
   <div class="row">
      <div class="col-lg-12">
                              <h2>Matches Played</h2>
         <div class="row py10">
            <!-- text box for day -->
            <div class="col-lg-2">
               <input placeholder="Day" type="number" min="1" max="31" class="form-
control" name="day" [(ngModel)]="day">
            </div>
            <!-- text box for month -->
            <div class="col-lg-2">
               <input placeholder="Month" type="number" min="1" max="12"
class="form-control" name="month" [(ngModel)]="month">
            </div>
            <!-- text box for year -->
            <div class="col-lg-2">
               <input placeholder="Year" type="number" min="1999" max="2030"
class="form-control" name="year" [(ngModel)]="year">
            </div>
            <div class="col-lg-2">
               <div class="">
                  <button class="GreenButton" (click)="searchByDate()">Search</button>
               </div>
```

```
            </div>
                                    <div class="col-lg-2">
              <div class="">
                                                    <button class="RedButton"
(click)="clear()">Clear Date</button>
              </div>
            </div>
          </div>

          <div class="table-responsive">
            <table class="table table-striped">
              <!-- table headings -->
              <thead>
                <tr>
                  <th>Date</th>
                  <th>Home Club</th>
                  <th>Home Score</th>
                  <th>Away Score</th>
                  <th>Away Club</th>
                </tr>
              </thead>
              <tbody>
                <!-- table data -->
                <tr *ngFor="let match of matches">
                  <td> {{ match.datePlayed}} </td>
                  <td> {{ match.homeTeam }} </td>
                  <td> {{ match.homeScore }} </td>
                  <td> {{ match.awayScore }} </td>
                  <td> {{ match.awayTeam }} </td>
                </tr>
              </tbody>

            </table>
          </div>

        </div>
      </div>
    </div>
```

### 4.6.2) third-gui.component.ts

```
import { Component, OnInit } from '@angular/core';
import { IMatch } from '../models';
import { DataService } from '../services/data.service';
import { DatePipe } from "@angular/common";

@Component({
```

```
 selector: 'app-third-gui',
 templateUrl: './third-gui.component.html',
 styleUrls: ['./third-gui.component.scss']
})
export class ThirdGuiComponent implements OnInit {

 matches: IMatch[] = [];
 day: number;
 month: number;
 year: number;

 constructor(private dataService: DataService) { }

 ngOnInit() {
  this.matchRequest();
 }

 //if the date is empty get all matches and if it is not empty try to get the matches by
date parameter
 searchByDate() {
        let date;
        if(!(this.day || this.month || this.year)){
                date = "";
        }
  else if(this.validateDate()){
                date = `${this.day}/${this.month}/${this.year}`;
        }
  this.matchRequest(date);
 }

 // Sending request to get matches by date
 private matchRequest(date: string = ""): void {
  this.dataService.sendGetRequest('match/matches', { date })
    .subscribe((data: IMatch[]) => {
     this.matches = data;
    }, (error) => {
     if (error.status === 409)
       alert(error.error)
    });
 }

 // Validation of fields day,month and year before sending the match request
 private validateDate(): boolean{
         let dateString = `${this.day}/${this.month}/${this.year}`;

         if(dateString.includes("undefined") || dateString.includes("null") ){
                 alert("Fill the rest of fields")
                 return false;
```

```
            }
            if(!(this.day >= 1 && this.day <= 31)){
                    alert("Days are between 1 and 31")
                    return false;
            }

            if(!(this.month >= 1 && this.month <= 12)){
                    alert("Months are between 1 and 12")
                    return false;
            }

            if(!(this.year >= 1993 && this.year <= 2040)){
                    alert("Years are between 1993 and 2040")
                    return false;
            }
            return true;
  }
  // clear all the text fields
  private clear():void{
            this.day = null;
            this.month = null;
            this.year = null;
  }
}
```

## 4.7) VIEW AND SORT MATCHES PLAYED WITH DATE (fourth-gui)

### 4.7.1) fourth-gui.component.html

```
<div class="container mt10">
   <div class="row">
      <div class="col-lg-12">
                  <h2>Matches Played</h2>
         <div class="py10 float-right">
            <button class="GreenButton" (click)="sort()">Sort in Ascending
order</button>
         </div>
         <div class="table-responsive">
            <table class="table table-striped">
               <!-- table headings -->
               <thead>
                  <tr>
                     <th>Date</th>
                     <th>Home Club</th>
                     <th>Home Score</th>
                     <th>Away Score</th>
```

```
                <th>Away Club</th>
              </tr>
            </thead>
            <tbody>
              <!-- table data -->
              <tr *ngFor="let match of matches">
                <td> {{ match.datePlayed}} </td>
                <td> {{ match.homeTeam }} </td>
                <td> {{ match.homeScore }} </td>
                <td> {{ match.awayScore }} </td>
                <td> {{ match.awayTeam }} </td>
              </tr>
            </tbody>

          </table>
        </div>

      </div>
    </div>
  </div>
```

### 4.7.2) fourth-gui.component.ts

```
import { Component, OnInit } from '@angular/core';
import { IMatch } from '../models';
import { DataService } from '../services/data.service';

@Component({
 selector: 'app-fourth-gui',
 templateUrl: './fourth-gui.component.html',
 styleUrls: ['./fourth-gui.component.scss']
})
export class FourthGuiComponent implements OnInit {

 matches : IMatch[] = [];

 constructor(private dataService: DataService) {
 }

 //Sending request to get all matches to display in data table
 ngOnInit() {
  this.dataService.sendGetRequest('match/matches').subscribe((data: IMatch[]) => {
   this.matches = data;
  });
 }
```

```
//Sending request to get all matches in ascending order of date
sort() {
   this.dataService.sendGetRequest('match/matches/order-by-date').subscribe((data:
IMatch[]) => {
     this.matches = data;
   });
  }
}
```

## 4.8) data.service.ts

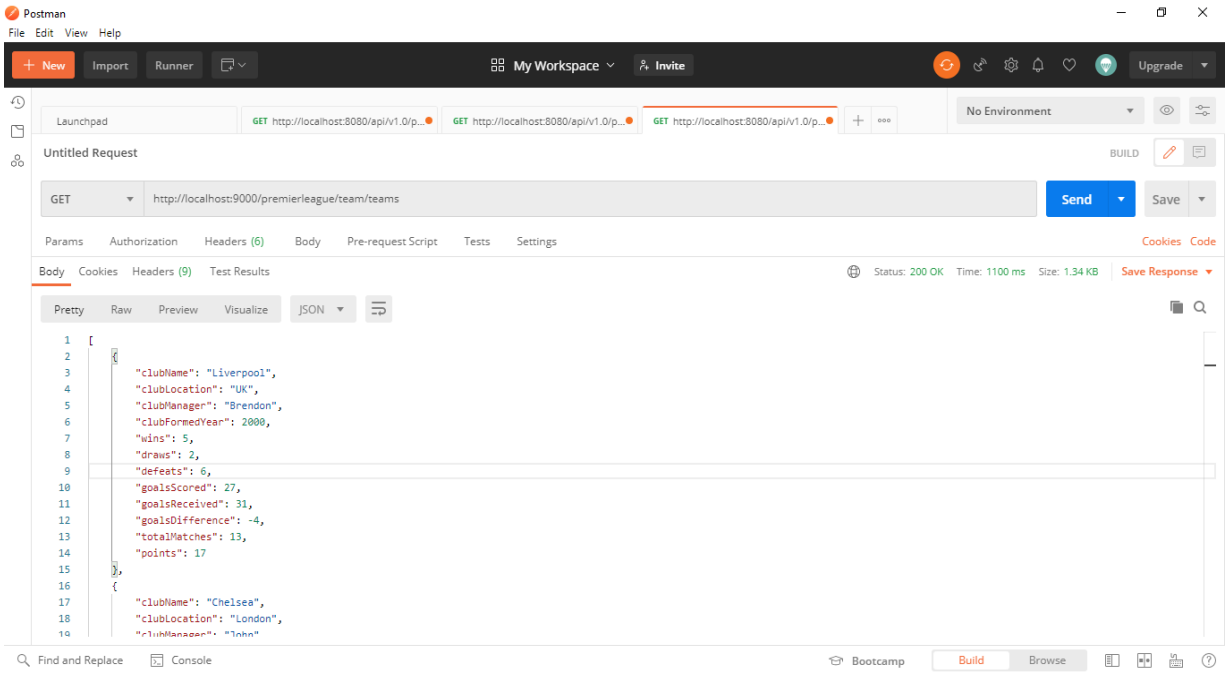
```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({
 providedIn: 'root'
})
export class DataService {

 private REST_API_SERVER = 'http://localhost:9000/premierleague/';

 constructor(private httpClient: HttpClient) {
 }
 //wrapping the Angular HTTP client get request
 public sendGetRequest(url: string, params?: any) {
   return this.httpClient.get(`${this.REST_API_SERVER}${url}`, {
     params: params
   });
 }
}
```

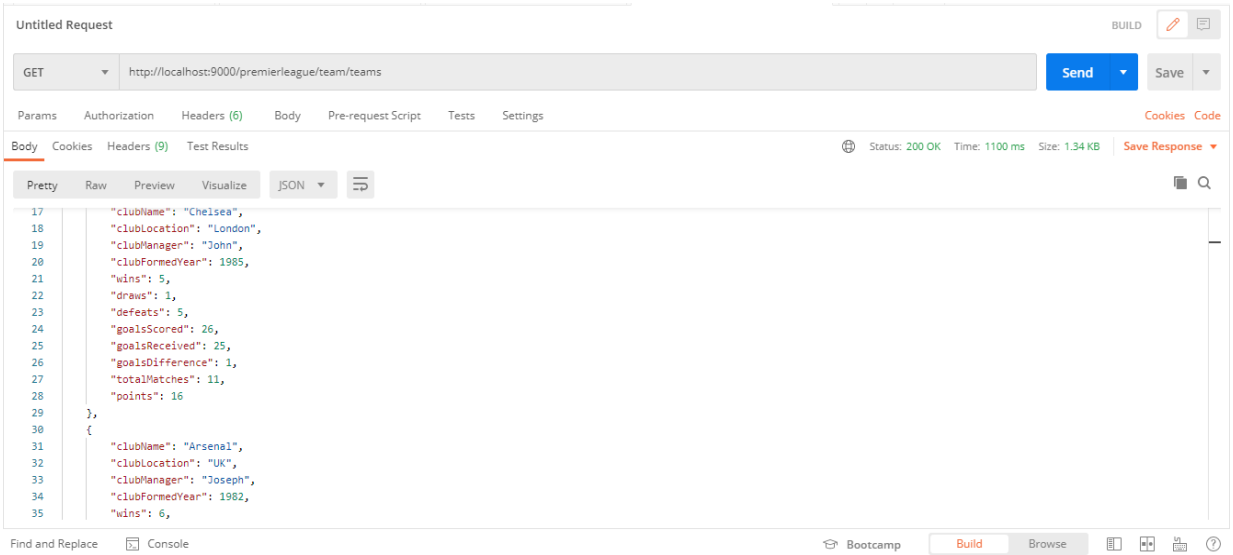
# 5) Routes – Play Framework

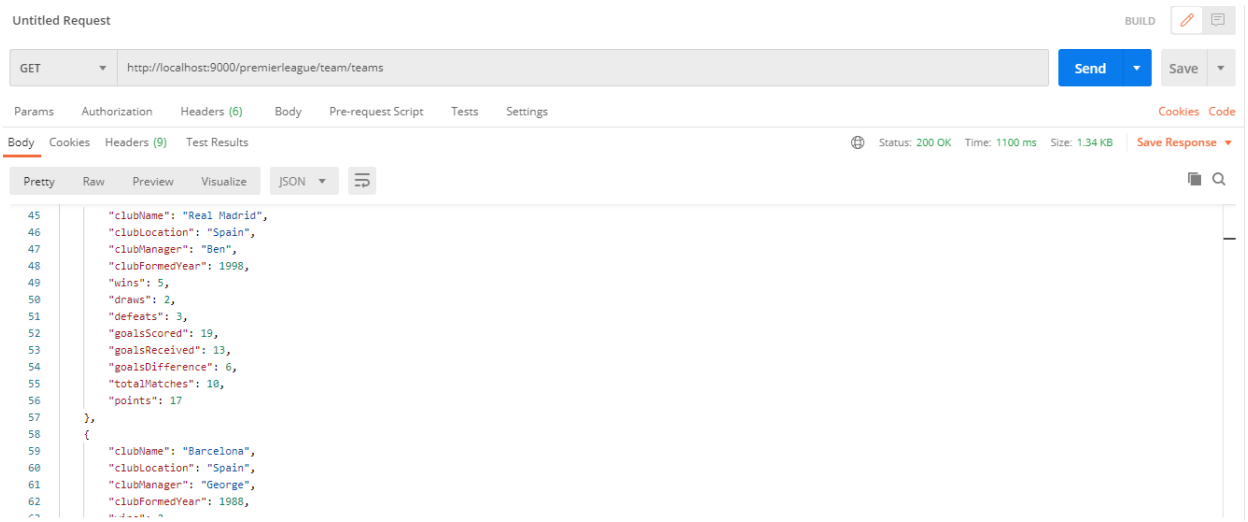
## 5.1)Screenshots of using Postman to test the REST APIs

1. # TEAMS

GET /premierleague/team/teams
controllers.team.TeamController.getTeams(orderBy:String ?= "")

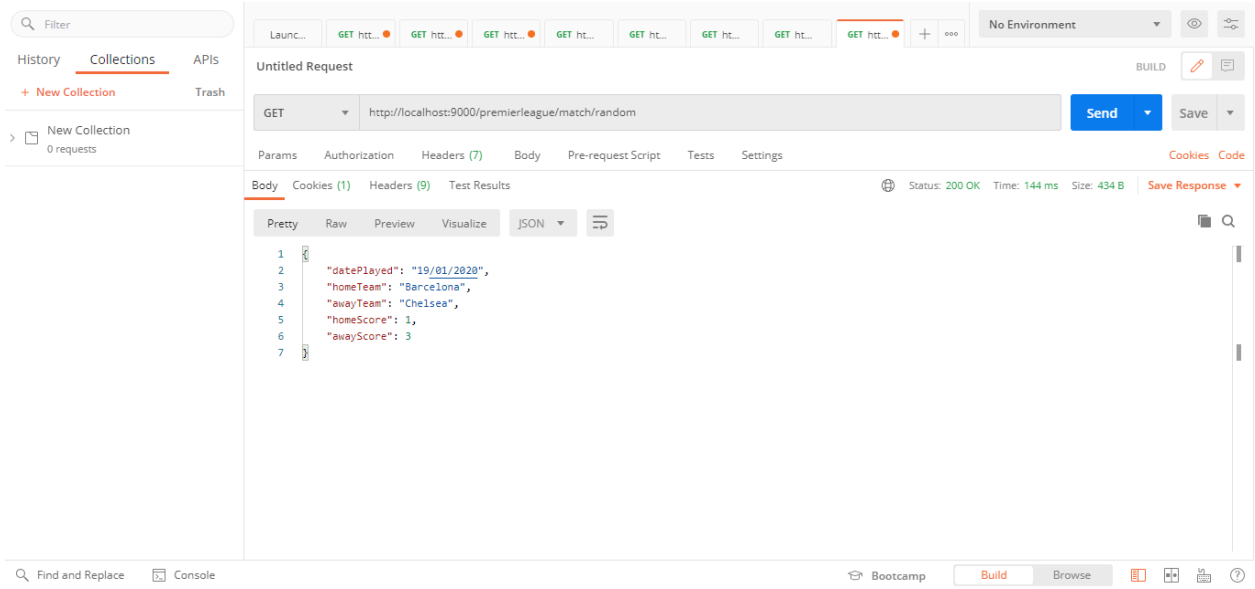





2. # MATCH

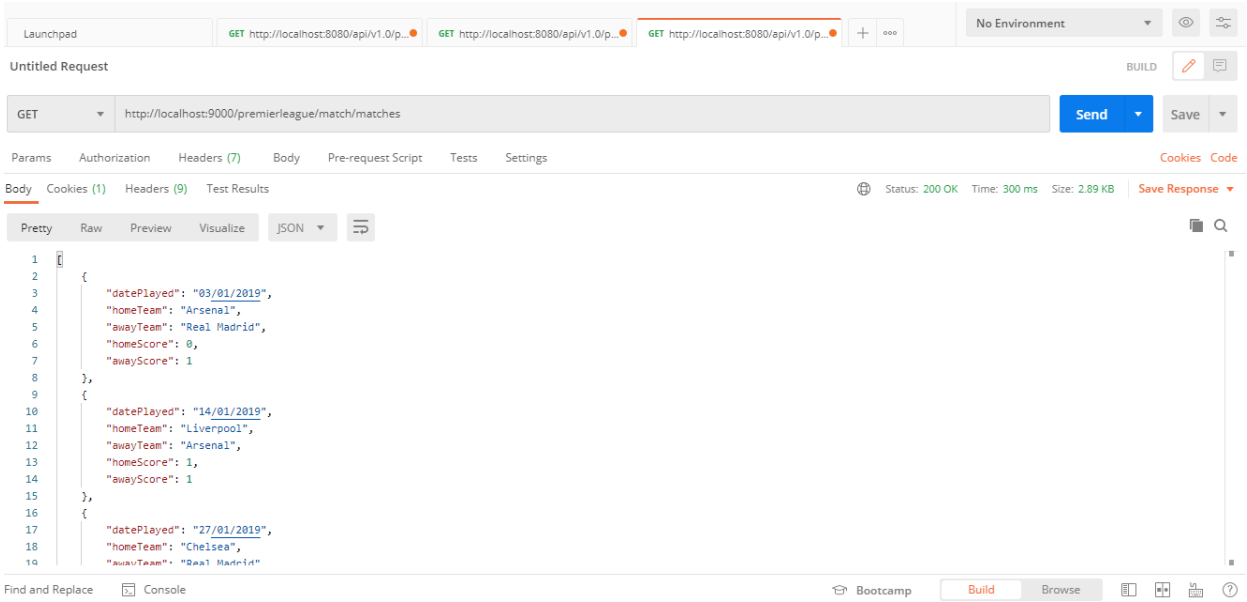
GET /premierleague/match/random
controllers.matches.MatchController.generateRandomMatch()



3.

GET /premierleague/match/matches

controllers.matches.MatchController.findMatchesByDate(date: String ?= "")




4.

GET /premierleague/match/matches/order-by-date

controllers.matches.MatchController.getMatchesOrderAscByDate()